

A PROJECT REPORT ON

SUSTAINABLE FINANCE

Dissertation Submitted

In partial fulfillment of the requirement for the award of the degree
of

BACHELOR OF COMPUTER APPLICATIONS

KERALA UNIVERSITY

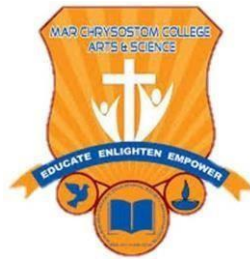
By:

AJIN VARUGHESE (Reg.No. - 33223157007)

FAHIM SHAMIM AZIZ (Reg.No. - 33223157017)

MIDHUN SUNIL (Reg.No. - 33223157022)

MUHAMMED ASLAM (Reg.No. - 33223157025)



MAR CHRYSOSTOM COLLEGE PARANTHAL - ADOOR

July 2025

**MAR CHRYSOSTOM COLLEGE
PARANTHAL, ADOOR**



CERTIFICATE

This is to Certify that this project report entitled “**SUSTAINABLE FINANCE**” is a Bonafide record of the project work done by,

AJIN VARUGHESE	(33223157007)
FAHIM SHAMIM AZIZ	(33223157017)
MIDHUN SUNIL	(33223157022)
MUHAMMED ASLAM	(33223157025)

Under our supervision and guidance, towards *partial fulfilment of the requirements for the award of the **Degree of BACHELOR OF COMPUTER APPLICATION*** under **University of Kerala** during the year 2023-2026

Ms. Saramma John

Project Guide

Ms. Saramma John

Head of Department

External Examiner

DECLARATION

We, **AJIN VARUGHESE (REG. No. 33223157007), FAHIM SHAMIM AZIZ (REG. No. 33223157017), MIDHUN SUNIL (REG. No. 33223157022), MUHAMMED ASLAM (REG. No. 33223157025)** declares that the project on “**SUSTAINABLE FINANCE**” is the result of original work done by us and to the best of our knowledge, a similar work has not been submitted earlier to the **University of Kerala** or any other Institution, for fulfillment of the requirement of a course of study.

This project report is submitted on partial fulfillment of the requirement for the degree of Bachelor of Computer Applications of University of Kerala

**AJIN VARUGEHSE
FAHIM SHAMIM AZIZ
MIDHUN SUNIL
MUHAMMED ASLAM**

ACKNOWLEDGEMENT

We with our utmost courteousness and gratitude submit our implacable praises to the **Omnipotent Almighty** for having bestowed his grace and blessings over us to accomplish this task of project dissertation.

We extend our sincere thanks to **Prof. GEORGI P DANIEL** (Principal, Mar Chrysostom College), **Ms. SARAMMA JOHN** (HOD- BCA Department, Mar Chrysostom College), **Ms. SARAMMA JOHN** (Internal Guide, Mar Chrysostom College) for their valuable advice, inspiration, whole hearted support and guidance at every aspect of this project.

We would like to express our sincere thanks to **ARCHON SOLUTIONS**, officials for providing us a chance to do project in their esteemed organization.

AJIN VARUGHESE
FAHIM SHAMIM AZIZ
MIDHUN SUNIL
MUHAMMED ASLAM

CONTENTS

	Page No
1. INTRODUCTION	1
1.1. ABOUT THE PROJECT	2
1.2. PROJECT OVERVIEW	3
1.3. ORGANIZATION PROFILE	4
2. SYSTEM STUDY AND ANALYSIS	6
2.1. REQUIREMENT ANALYSIS	8
2.2. EXISTING SYSTEM	9
2.3. PROPOSED SYSTEM	9
2.4. FEASIBILITY STUDY	10
2.4.1. OPERATIONAL FEASIBILITY	10
2.4.2. TECHNICAL FEASIBILITY	11
2.4.3. ECONOMICAL FEASIBILITY	11
3. REQUIREMENT SPECIFICATION	12
3.1. HARDWARE SPECIFICAITON	13
3.2. SOFTWARE REQUIREMENTS	14
3.3. SOFTWARE DESCRIPTION	14
3.4. FUNCTIONAL REQUIREMENTS	19
4. SYSTEM DESIGN	20
4.1. DATABASE DESIGN	22
4.1.1. DATA FLOW DIAGRAM	33
4.1.2. USE CASE DIAGRAM	38
4.1.3. FLOW CHART	42
4.2. PROCEDURAL DESIGN	43
4.3. INTERFACE DESIGN	43
4.4. INPUT DESIGN	43
4.5. OUTPUT DESIGN	44
5. SYSTEM CODING	45
6. SYSTEM TESTING	58
6.1. TESTING	59
6.2. UNIT TESTING	59
6.3. INTEGRATION TESTING	60
6.4. SYSTEM TESTING	61
6.5. USER ACCEPTANCE TESTING	61
7. SYSTEM IMPLEMENTATION AND MAINTENANCE	63
7.1. SYSTEM IMPLEMENTATION	64
7.2. SYSTEM MAINTENANCE	65
7.3. FUTURE SCOPE & FUTURE ENHANCEMENT	65
8. CONCLUSION	67
9. APPENDIX	69
9.1. SCREEN LAYOUTS AND REPORTS	70
10. BIBLIOGRAPHY	76

INTRODUCTION

1. INTRODUCTION

1.1 ABOUT THE PROJECT

The **Sustainable Finance** Eligibility System is a rule-based digital platform designed to streamline and standardize the evaluation of loan applications for sustainable development projects. This system enables banks to create dynamic questionnaires aligned with key environmental themes such as Pollution Prevention, Water Management, Circular Economy, Climate Adaptation, Climate Mitigation, and Biodiversity.

Each questionnaire contains multiple-choice questions where each choice carries a specific score, configurable by bank administrators. These questions are categorized according to the applicant's selected loan type or sustainability focus area. Applicants respond to the questionnaire based on their project's characteristics, and the system automatically calculates their total sustainability score.

The platform uses predefined eligibility rules to determine whether an applicant qualifies for sustainability-linked financial support. The decision-making process is entirely automated, ensuring objectivity, consistency, and transparency in the loan pre-qualification process. Additionally, applicants are required to upload a project report document, which is stored securely and made accessible to both the applicant and the reviewing bank officers.

The system also provides an admin interface for managing questions, categories, and scoring logic without requiring any technical knowledge. By integrating automation and sustainability principles, the platform simplifies the eligibility process and encourages environmentally responsible lending practices.

1.2 PROJECT OVERVIEW

“**Sustainable Finance**” is a cloud-based web application designed to streamline the evaluation and approval of green loans for individuals, enterprises, and governments. Built using Java 17, Spring Boot, Maven, and Hibernate for the backend, with ReactJS for the frontend and MySQL for database management, the system ensures scalability, security, and accessibility. The platform follows Agile methodology with GitHub for version control, enabling collaborative and iterative development.

The application automates the loan pre-qualification process through dynamic questionnaires and a rule-based engine aligned with sustainability standards such as the EU Taxonomy. Applicants can submit project details and upload reports, while the system calculates sustainability scores, validates eligibility, and provides real-time feedback. Role-based access ensures transparency and secure management across applicants, bank officers, and administrators.

Core Features:

1. **Dynamic Questionnaire Management:**
 - Banks create customizable questionnaires aligned with sustainability themes (e.g., Climate Mitigation, Water Management, Biodiversity).
 - Choices carry scores, and responses automatically compute sustainability ratings.
2. **Automated Eligibility & Rules Engine:**
 - Predefined rules map responses to green loan eligibility.
 - Rules can be dynamically updated via an admin interface without coding.
3. **Applicant Module:**
 - Registration and loan application submission with project report upload.
 - Real-time feedback on eligibility and status tracking.
4. **Bank Module:**
 - Loan application review, updates, and disbursement tracking.
 - Portfolio monitoring of sustainability-linked loans.
5. **Admin Module:**
 - Management of eligibility rules, questions, and sustainability scoring.
 - Complaint handling and system oversight.

1.3 ORGANIZATION PROFILE

Archon Solutions Pvt. Ltd Founded in 2013 in Kochi, India, Archon Solutions Pvt. Ltd. is a leading global IT consultancy and software solution provider, delivering advanced digital solutions that empower organizations to thrive in a rapidly changing technology landscape.

Since our inception, we have delivered a comprehensive range of custom software development services, enabling businesses to remain competitive in today's fast-paced world of digital transformation and innovation. We are driven by a passion for building intelligent, efficient, and forward-thinking technologies that meet the evolving needs of modern enterprises.

We specialize in crafting end-to-end software solutions tailored to address complex business challenges. Our offerings include in-house development projects and institutional programs, all designed to enhance operational efficiency, drive innovation, and support long-term strategic goals.

As part of our global expansion, we have established branches in Australia, Belgium, and the United Kingdom, while continuing to grow and innovate from our foundation in Kochi, India. Archon Solutions is a dynamic and innovative technology company specializing in providing software solutions tailored to meet the unique needs of its clients. With a focus on cutting-edge technologies and a commitment to customer satisfaction, Archon Solutions aims to deliver high-quality products and services that drive business growth and efficiency.

Mission Statement:

Our mission at Archon Solutions is to deliver transformative software solutions that enhance operational efficiency, drive innovation, and support strategic goals.

We are committed to providing exceptional value through integrity, excellence, and long-term partnership.

Core Values:

- **Innovation**
Continuously evolving to create smart, future-ready solutions
- **Integrity**
Operating with transparency, accountability, and ethical commitment
- **Customer-Centricity**
Putting client satisfaction at the core of every engagement

- **Collaboration**

Building meaningful partnerships through teamwork and communication

- **Excellence**

Striving for the highest quality in every service we deliver

Services:

- **Custom Software Development**

Personalized software solutions tailored to meet unique business needs

- **Web and Mobile Application Development**

Scalable, user-friendly applications for dynamic digital experiences

- **Enterprise Software Solutions**

Integrated systems to optimize and streamline business processes

- **Cloud Computing Services**

Flexible and scalable cloud platforms to support business growth

SYSTEM STUDY AND ANALYSIS

2. SYSTEM ANALYSIS

System analysis is a detailed study of various operations performed by a system and their relationship within and outside of the system. Here the key question is: What must be done to solve the problem? One aspect of analysis is defining the boundaries of a system and determines whether or not the candidate system should consider other related systems. Analysis begins when a user or manager begins a study of the program using the existing system.

During analysis, data is collected on the various files, decision points and transactions handled by the present system. The commonly used tools in system analysis are dataflow diagram, interviews, onsite observation etc. System analysis is application of the system approach to the problem-solving using computers. The ingredients are system elements, process and technology. This means that to do system work, one is to understand the system concepts and how the organizations operate as a system and the design appropriate computer-based system that will meet the organizations requirements. It is actually customized approach to the use of computer problem solving.

Analysis can be defined as the separation of a substance into parts for study an interpretation, detailed examination. System development revolves around a lifecycle that begins with the recognition of user needs. The critical phase of managing system project is planning. To launch a system investigation, we need a master plan detailing the steps taken, the people to be questioned and outcome expected.

System analysis can be categorized into four parts:

- System planning and initial investigation.
- Information Gathering.
- Applying analyzing tools for structured analysis.
- Feasibility study.
- Cost/Benefit analysis.

System study or system analysis is the first among the four-life cycle phase of a system. System begins when a user or manager requests a studying of a program in either an existing or a project one. It involves studying the base of the organizations currently operating, retrieving and processing data to produce information with the goal of determining how to make it work better. System analysis itself breaks down into stages-Preliminary and Detailed. During

preliminary analysis the analyst and the user list the objectives of the system. To arrive at a preliminary report, the analyst interviews key personnel in the organization and schedule meetings with the users and the management.

Thus, the objective of analysis phase of the system–analysis and design exercise is the establishment of the requirement for the system to be acquired, developed and installed. Fact-finding or gathering is essential to any analysis of requirements. In brief analysis of the system helps an analyst to make a clear view of the existing system and thereby can give suggestions for the improvement of the new system. Information's about the organization's policies, goals, objectives, and structure explains the kind of environment that promotes the introduction of computer-based system. It is necessary that the analyst must be familiar with the objectives, activities and the functions of the organizations in which the computer system is to be implemented.

2.1 REQUIREMENT ANALYSIS

Requirement Analysis, also known as Requirement Engineering, is the process of defining user expectations for a new. software being built or modified. In software engineering, it is sometimes referred to loosely by names such as requirements gathering or requirements capturing.

REQUIREMENT ANALYSIS PROCEDURE

1. Document analysis. User manuals and process documents about the current system can become helpful in defining software requirements.
2. Interview
3. Observation
4. Workshop
5. Brainstorming
6. Prototyping
7. Collect and Understand Requirements
8. Define Requirements

2.2 EXISTING SYSTEM

The current loan evaluation systems used by banks and financial institutions largely rely on manual processes, static eligibility checklists, and paper-based forms. Loan officers assess an applicant's eligibility based on traditional financial parameters, often without structured mechanisms for evaluating sustainability factors. There is minimal use of automation or intelligence in screening loan applications against environmental criteria such as carbon impact or alignment with green project categories. Moreover, there is no centralized system to dynamically manage eligibility rules, categorize loans by type, or generate impact reports. This leads to inconsistencies, delayed decisions, limited transparency, and difficulty in tracking sustainability performance across loan portfolios.

2.3 PROPOSED SYSTEM

The proposed Sustainable Finance system transforms the traditional loan evaluation process by introducing a smart, automated, and sustainability-aware platform. It utilizes structured questionnaires, a dynamic rule engine, and EU Taxonomy-based activity mapping to assess whether a loan application supports sustainable development goals. Administrators can easily define and update eligibility criteria, categorize questions based on loan types, and match responses to predefined taxonomy activities. The system automates loan pre-screening, validates environmental compliance, and provides real-time feedback to users. It also supports detailed analytics and reporting on sustainability impact, helping banks monitor green loan portfolios effectively.

ADVANTAGES

- Automation of Loan Eligibility Evaluation
- Alignment with EU Taxonomy for Sustainability
- Dynamic Questionnaires Based on Loan Categories
- Real-Time Eligibility Feedback for Applicants
- Admin Interface for Rule & Question Management
- Carbon Impact & Green Metrics Tracking
- Transparent, Objective & Scalable Assessment System
- Secure Role-Based Access for Admins, Officers, and Applicants

2.4 FEASIBILITY STUDY

An initial investigation culminates in a proposal that determines whether an ultimate system is feasible. When a proposed system is made and approved it initiates a feasibility study. The purpose of the feasibility study is to identify various candidate systems and evaluate whether they are feasible by considering technical, economical, and operational feasibility and to recommend the best candidate system. The feasibility of such a program is listed in a simulated environment. Once all features are working properly in a simulated environment, we can implement them in a real platform.

A feasibility study aims to objectively and rationally uncover the strengths and weaknesses of an existing business or proposed venture, opportunities and threats present in the environment, the resources required to carry through, and ultimately the prospect for the successes. In the simplest terms the two criteria to judge feasibility are cost required and value to be attained. During product engineering, we considered the following types of feasibility. The study is done in six phases.

- Operational feasibility
- Technical feasibility
- Economic feasibility

2.4.1 OPERATIONAL FEASIBILITY

The system is designed with intuitive interfaces and a role-based access model, ensuring ease of use for administrators, loan applicants, and bank officers. Users can seamlessly interact with the platform to create questionnaires, respond to questions, and evaluate loan eligibility. The admin panel allows non-technical staff to manage rules and taxonomy without coding. Automated feedback and analytics reduce manual workload and increase decision-making speed. Because the system follows familiar patterns like form-based input, rule-based validation, and dashboard reporting, the learning curve is minimal, and integration with existing loan processes is smooth.

2.4.2 TECHNICAL FEASIBILITY

The proposed **Sustainable Finance** system is technically feasible due to the availability of mature, reliable, and well-supported development frameworks and technologies. The backend is developed using Java Spring Boot, ensuring scalability, modularity, and security. The frontend leverages ReactJS, providing a responsive and user-friendly interface. Database management is handled using MySQL, while Spring Data JPA enables efficient ORM-based data handling. The system integrates sustainability taxonomy data, rule engines, and real-time validations — all of which can be implemented using existing open-source libraries and tools. The project is deployable on cloud platforms (e.g., AWS, Azure), making it future-proof and easily scalable.

2.4.3 ECONOMICAL FEASIBILITY

Economically, the system provides a high return on investment (ROI) by significantly reducing manual evaluation costs, minimizing paper-based workflows, and improving turnaround time for loan processing. It reduces the need for additional staff or consultants to perform sustainability assessments. Since it uses open-source technologies, licensing costs are minimal. The long-term cost savings from automation, better decision accuracy, and compliance with green financing norms make the project financially viable. Moreover, institutions adopting this system may qualify for green finance incentives or ESG ratings, further enhancing its value.

REQUIREMENT SPECIFICATION

3. SOFTWARE SPECIFICATION

A software requirements specification (SRS) is a detailed description of a software system to be developed with its functional and non-functional requirements. The SRS is developed based on the agreement between customer and contractors. It may include the use cases of how the user is going to interact with software system. The software requirement specification document is consistent of all necessary requirements required for project development.

3.1 HARDWARE SPECIFICATION

The hardware for the system is selected considering the factors such as CPU processing speed, memory access speed, peripheral channel speed, printer speed; seek time & relational delay of hard disk and communication speed etc.

The hardware specifications are as follows:

Processor : Intel Pentium Quad Core

Operating Speed : 2.66GHZ

Memory

RAM : 2GB

Disk Storage

Hard Disk : 500GB

Data Backup Device : Pen drive

Peripherals

Keyboards : 106 keys

Mouse : Standard

Monitors : Proper Resolution

Printers : Laser/Inkjet

3.2 SOFTWARE REQUIREMENTS

The software requirements for the Sustainable Finance system have been carefully chosen to ensure scalability, security, platform independence, and developer productivity. Modern and widely adopted technologies have been selected to facilitate long-term maintainability, performance, and integration with cloud services.

Component Specification

Operating System	:	Windows 10 / 11, Linux (Ubuntu recommended)
Frontend	:	ReactJS, HTML5, CSS3, JavaScript
Backend	:	Java 17+, Spring Boot (REST API)
Database	:	MySQL
ORM	:	Spring Data JPA with Hibernate
Authentication	:	Spring Security with JWT
IDE / Editor	:	IntelliJ IDEA / Visual Studio Code
Build Tool	:	Maven
Browser	:	Google Chrome, Mozilla Firefox, Edge
API Testing Tool	:	Postman
Version Control	:	Git with GitHub

3.3 SOFTWARE DESCRIPTION

Windows 10/11

An Operating System may be viewed as an organized collection of software extensions of hardware, consisting of control routines for operating a computer and for providing an environment for execution of programs. Programs rely on facilities provided by the operating system to gain access to computer-system resources such as files and input/output devices, i.e. the operating system acts as interface between users and the hardware of a computer system.

Windows 10

It is a major release of Microsoft's Windows NT operating system. It is the direct successor to Windows 8.1, which was released nearly two years earlier. It was released to manufacturing on July 15, 2015, and later to retail on July 29, 2015. Windows 10 is the final version of Windows that supports 32-bit processors (IA-32 and ARMv7-based) and devices with BIOS firmware.

Its successor, Windows 11, requires a device that uses UEFI firmware and a 64-bit processor in any supported architecture (x86-64 for x86 and ARMv8 for ARM). Windows 10 makes its user experience and functionality more consistent between different classes of device and addresses many shortcomings of the user interface introduced in Windows 8. Windows 10 Mobile, the successor to Windows Phone 8.1, shared some user interface elements and apps with its PC counterpart.

Windows 10 supports universal apps, an expansion of the Metro-style first introduced in Windows 8. Universal apps can be designed to run across multiple Microsoft product families with nearly identical code—including PCs, tablets, smartphones, embedded systems, Xbox One, Surface Hub and Mixed Reality. The Windows user interface was revised to handle transitions between a mouse-oriented interface and a touchscreen-optimized interface based on available input devices—particularly on 2-in-1 PCs. Both interfaces include an updated Start menu which incorporates elements of Windows 7's traditional Start menu with the tiles of Windows 8. Windows 10 also introduced the Microsoft Edge web browser, a virtual desktop system, a window and desktop management feature called Task View, support for fingerprint and face recognition login, new security features for enterprise environments, and DirectX 12.

Overview of JAVA SPRINGBOOT

The Spring Boot is a powerful, open-source, Java-based framework designed to simplify and accelerate the development of standalone, production-grade web applications. It is part of the larger Spring ecosystem and was created by Pivotal Software to reduce the complexity of configuration and deployment in Java enterprise development. Spring Boot follows a microservice-friendly architecture and allows developers to focus on writing business logic without having to worry about boilerplate code or complex setup.

Spring Boot is built on top of the traditional Spring Framework, offering features like auto-configuration, embedded servers (Tomcat/Jetty), and seamless integration with technologies like Spring Data JPA, Spring Security, and RESTful APIs. It supports the MVC (Model-View-Controller) architecture and encourages clean separation between data, business logic, and presentation layers. With support for dependency injection, robust security, easy database connectivity, and cloud-native capabilities, Spring Boot is widely used in enterprise-grade applications and is ideal for building scalable, modular, and maintainable web systems.

Key Features of Spring Boot:

- **Auto-Configuration:** Automatically configures application components based on included dependencies, reducing manual setup.
- **Embedded Web Servers:** Comes with built-in Tomcat/Jetty/Undertow servers for running applications independently—no need for external server deployment.
- **Spring MVC Support:** Allows the development of REST APIs and web services using the Model-View-Controller architecture.
- **Spring Data JPA:** Simplifies database operations using object-relational mapping (ORM) and repository abstractions.
- **Security Integration:** Offers seamless integration with Spring Security for implementing authentication and authorization (e.g., with JWT tokens).
- **Dependency Injection (DI):** Encourages loosely coupled code and improved testability through inversion of control.
- **Actuator:** Provides production-ready features like health checks, metrics, and monitoring endpoints.
- **Custom Configuration:** Supports flexible configurations using properties or .yaml files and `@Value` or `@ConfigurationProperties` annotations.
- **Microservice Ready:** Supports cloud deployment, service discovery, and scalability—ideal for distributed systems.

MySQL

MySQL is a widely-used, open-source relational database management system (RDBMS) that supports a broad range of applications from small to large-scale web applications. Unlike SQLite, MySQL operates with a client-server architecture, where the database is hosted on a server and accessed via a network connection. MySQL supports a full implementation of SQL, providing a robust, scalable system with extensive features such as stored procedures, triggers, and user-defined functions. It is ACID-compliant, ensuring reliable transactions, and it offers excellent performance for high-concurrency write operations and complex query processing. MySQL is known for its ability to handle high-traffic websites and large volumes of data with ease. While MySQL is typically more complex to set up and maintain than SQLite, it is highly suited for large-scale, high-traffic applications and systems that require advanced database management capabilities.

ReactJS

JavaScript ReactJS is a JavaScript library for building user interfaces, primarily for single-page applications where you need a fast and dynamic response to user interactions. ReactJS simplifies the process of creating interactive UIs by allowing developers to build reusable UI components. Unlike traditional JavaScript, React uses a declarative syntax and a virtual DOM for efficient updates and rendering. ReactJS is maintained by Facebook and has become one of the most popular libraries for front-end development due to its simplicity, performance, and flexibility. ReactJS code can be written using the .jsx extension, which allows you to mix JavaScript and HTML-like syntax (JSX). This makes React code more readable and easier to write compared to traditional JavaScript. ReactJS is client-side, running directly in the browser, but it can also be used in server-side rendering with frameworks like Next.js. React components can manage their own state and can be easily integrated with other libraries or APIs. ReactJS is widely compatible with modern web browsers, and it allows seamless interaction with HTML elements, such as handling form inputs, button clicks, and other user actions. ReactJS is also used in mobile app development through React Native. React's component-based structure enables you to break down complex UIs into smaller, reusable parts, making development faster and easier. It provides powerful features like hooks, lifecycle methods, and state management, which allow you to manage and manipulate data within your components. ReactJS is highly scalable and ideal for dynamic, high-performance web applications.

CSS

CSS (Cascading Style Sheets) is used to control the style and layout of a web document in a simple and efficient way. CSS defines the presentation of HTML elements, handling aspects such as color, fonts, spacing, and positioning. Unlike HTML, which structures the content, CSS enhances the visual appearance of a webpage. CSS allows you to apply styles across an entire website, making it easier to maintain a consistent design. It supports responsive design, enabling web pages to adapt to different screen sizes using media queries. With CSS, you can create layouts using techniques like Flexbox or CSS Grid, allowing for flexible and dynamic designs. CSS is fundamental for making websites visually appealing and ensuring a great user experience across devices.

HTML

HTML stands for Hyper Text Mark-up Language, which is the most widely used language on Web to develop web pages. HTML was created by Berners-Lee in late 1991 but "HTML 2.0" was the first standard HTML specification which was published in 1995. HTML 4.01 was a major version of HTML and it was published in late 1999. Though HTML 4.01 version is widely used but currently we are having HTML5 version which is an extension to HTML 4.01, and this version was published in 2012. Originally, HTML was developed with the intent of defining the structure of documents like headings, paragraphs, lists, and so forth to facilitate the sharing of scientific information between researchers. Now, HTML is being widely used to format web pages with the help of different tags available in HTML language.

VISUAL STUDIO CODE

Visual Studio Code (VS Code) is a lightweight, open-source source code editor developed by Microsoft, designed to work with various programming languages, including JavaScript and React. Built on the Electron framework, VS Code is available for Windows, Linux, and macOS. It offers essential features such as syntax highlighting, intelligent code completion, debugging support, code snippets, and Git integration. The editor is highly customizable, allowing users to change themes, keyboard shortcuts, and install extensions to enhance functionality. VS Code is an ideal tool for React development, providing a streamlined and efficient environment for building web applications.

INTELLIJ

IntelliJ IDEA is a powerful IDE that is widely used for Java development, particularly for building Spring Boot applications. It provides features such as advanced code refactoring, smart code completion, real-time debugging, and integration with build tools like Maven and Gradle. IntelliJ IDEA offers robust support for Spring Boot, making it a preferred choice for developers working on enterprise-level Java applications. With a focus on productivity and user experience, IntelliJ ensures that developers can quickly build, test, and deploy Spring Boot applications with ease.

3.4 FUNCTIONAL REQUIREMENTS

The **Sustainable Finance** is an advanced banking software solution designed to assist individuals, enterprises, and governments in securing loans for sustainable development projects. Built with **Java Spring Boot** for the backend and **ReactJS** for the frontend, this system supports a cloud-based deployment model, ensuring scalability, performance, and accessibility. The software focuses on providing green loans at reduced interest rates, encouraging investments in environmentally sustainable initiatives.

Automated Grading System with College Handling provides the modules like

- ❖ Applicant Module
- ❖ Bank Finance Module
- ❖ Admin Module

Applicant Module

- ❖ Registration
- ❖ Questionnaire answering
- ❖ Loan application creating
- ❖ Loan application viewing
- ❖ Loan application updating

Bank Module

- ❖ Questionnaire adding
- ❖ Loan application viewing
- ❖ Loan application updating
- ❖ Disbursement updating
- ❖ Payment updating

Admin Module

- ❖ Rules managing
- ❖ Complaints handling

SYSTEM DESIGN

4. SYSTEM DESIGN

Design of a system can be defined as the process of applying various techniques and principles for the purpose of defining a device, a process or system is sufficient details to permit its physical realization. Thus, system design is a solution a “how to” approach to the creation of a new system. This important phase provides the understanding and procedural details necessary for implementing the system recommended in the feasibility study.

The data design transforms the information domain model created during analysis into the data structure that will be required to implement the software. The architectural design defines the relationship among major structural components into a procedural detail necessary for implementing the system recommended in the feasibility study.

The data design transforms the information domain model created during analysis into the data structure that will be required to implement the software. The architectural design defines the relationship among major structural components into a procedural description of the software. Source code is generated and testing is conducted to integrate and validate the software. Project management point of view software design is conducted in two into data and software architecture. There are two levels of the system design:

- Logical design
- Physical design

In the logical design, the designer produces a specification of the major features of the system which meets the objectives. The delivered product of logical design includes current requirements of the following system components:

- Input design.
- Output design.
- Database design.

Physical design takes this logical design blueprint and produces the program software, files and a working system. Design specifications instruct programmers about what the system should do. The programmers in turn write the programs that accept input from users, process data, produce reports, and store data in files. Structured design is a data flow-based methodology that partitions a program into a hierarchy of modules organized in a top-down manner with details.

4.1 DATABASE DESIGN

A database is a collection of interrelated data stored with minimum redundancy to serve users more quickly and efficiently. James Martin defines database as "A collection of data designed to be used by different programs". The general objective of a database is to make information access easy, quick, inexpensive, integrated, and shared by different applications and users.

Database design is an important yet sometimes overlooked part of the application development lifecycle. An accurate and up-to-date data model can serve as an important reference tool for Database Administrators, developers, and other members of joint application development team.

A good database design does the following:

- Provide minimum search time when locating specific records.
- Stored data in the most efficient manner possible to keep the database from
- Growing too large
- Make data update as easy as possible.
- Is flexible enough to allow inclusion of new functions required of the programs.

The database design is a two-level process. In the first step user requirements are gathered together and a database is designed which will meet these requirements as cleanly as possible. This step is called Information Level Design, and it is taken independent of any individual DBMS.

In the second step this information level design is transferred into a design for the specific DBMS that will be used to implement the system in question. This step is called Physical Level Design, concerned with the characteristics of the specific DBMS that will be used. A database design runs parallel with the system design. The organization of the data in the database is aimed to achieve two major objectives, the two objectives are:

- Data Integrity
- Data independence

The data base design is made up of three levels:

- Conceptual level (High level)
- Physical level (Low level)
- View level (Representation level)

Conceptual Level

Conceptual level describes the essential features of the system data. It uses symbols and is called entity-relationship analysis. An entity is a conceptual representation of an object.

Physical Level

In this level data is stored physically. That is an internal schema that describes the physical storage structure of the database.

View Level

This level is used to describe how the user views the records or objects in the database. Data structuring is refined through a process called normalization. Data is grouped into the simplest way possible, so that later changes can be made with a minimum impact on data structures. Based on the requirements determined during the definition phase of project life cycle, the data elements describing the entity were determined. They are later submitted to normalization to remove redundancies and to optimize them.

The concepts and techniques used when designing an effective database includes:

An **entity** is a logical collection of things that are relevant to a database. The physical counterpart of an entity is a database table.

An **attribute** is a descriptive or quantitative characteristic of an entity. The physical counterpart of an attribute is a database column (or field).

A **primary key** is an attribute (or combination of attributes) that uniquely identifies each instance of an entity. A primary key cannot be null, and the value assigned to a primary key should not change over time. A primary key also needs to be efficient. For example, a primary key that is associated with an INTEGER data type will be more efficient than one that is associated with a CHAR data type.

A **relationship** is a logical link between two entities. A relationship represents a business rule and can be expressed as a verb phrase.

A **foreign key** exists when the primary key of a parent entity exists in a child entity. A foreign key requires that values must be present in the parent entity before like values may be inserted in the child entity.

Normalization

Normalization is the process of efficiently organizing data in a database. There are two goals of the normalization process: eliminate redundant data (for example, storing the same data in more than one table) and ensure data dependencies make sense (only storing related data in a table). Both if these are worthily goals as they reduce the amount of space a database consumes and ensure that data is logically stored.

The process of refining the data model and creating it with a database is called normalization. Database designers can implement several levels of normalization. Each level builds on the previous level by reducing the amount of redundancy between tables. This typically increases performance and avoids problems with data consistency.

1. First Normal Form

- First normal form is the most basic level of database normalization. The keys to creating tables in the 1NF.
- Eliminate repeating groups in individual tables.
- Create a separate table for each set of related data.
- Identify each set of related data with a primary key.

2. Second Normal Form

The first normal form is not good database design, but it is a start. The keys to creating tables in the 2NF create separate tables for set of values that apply to multiple records.

3. Third Normal Form

The third normal form is typically the last form that most database designers will normalize to. After 3NF, the number of relationships between tables becomes very large, and performance begins to decrease on the database server. The goal of the third normal form is to remove all data that does not depend on the primary key of table.

Advantages of normalization are:

- Help in reduction in the complexity of maintaining data integrity by removing the redundant data.
- It reduces inconsistency of data.
- Eliminate the repeating fields

TABLES

Table name: answer

Purpose: To store answer from questionnaire.

Column	Type	Size	Constraints	Description
id	int	11	Primary key	To store answer id
created_at	datetime	6	Not Null	To store when answer is created
updated_at	datetime	6	Not Null	To store when answer is updated
answer_text	varchar	255	Not Null	To store answer text
choice_d	int	40	Not Null	To store choice id
user_id	int	10	Not Null	To store user id
user_id	bigint	11	Foreign key	To store user id

Table name: choice

Purpose: To store choice details.

Column	Type	Size	Constraints	Description
id	int	11	Primary key	To store choice id
created_at	datetime	6	Not Null	To store when choice is created
updated_at	datetime	6	Not Null	To store when choice is updated
choice_text	varchar	255	Not Null	To store choice text
score	varchar	40	Not Null	To store score
question_id	bigint	11	Not Null	To store question id
question_id	bigint	11	Foreign key	To store question id

Table name: feedback

Purpose: To store feedback details.

Column	Type	Size	Constraints	Description
id	int	11	Primary key	To store feedback id
created_at	datetime	6	Not Null	To store when feedback is created
updated_at	datetime	6	Not Null	To store when feedback is updated
description	text	75	Not null	To store description
document_url	varchar	255	Not null	To store document url
title	text	75	Not null	To store title
user_id	int	11	Not null	To store user id
user_id	int	11	Foreign key	To store user id

Table name:loan_category

Purpose: To store the loan categories.

Column	Type	Size	Constraints	Description
id	int	11	Primary key	To store loan category id
loan_category	varchar	100	Not Null	To store loan category
Interest_rate	double	8	Not Null	To store interest rate
created_at	datetime	6	Not Null	To store when loan category is created
updated_at	datetime	6	Not Null	To store when loan category is updated

Table name: loan

Purpose: To store loan details

Column	Type	Size	Constraints	Description
id	int	11	Primary key	To store loan id
created_at	datetime	6	Not Null	To store when loan is created
updated_at	datetime	6	Not Null	To store when loan is updated
amount	double	8	Not null	To store how the amount is
amount_pending	double	8	Not null	To store amount is pending
bank_accepted	bit	1	Not null	To store loan is bank accepted
duration_month	bigint	11	Not null	To store duration of months
interest_rate	double	8	Not null	To store interest rate of loan
loan_category	enum	255	Not null	To store loan category
loan-start_date	datetime	6	Not null	To store when the loan is started
loan_url	binary	16	Not null	To store url of loan
project_name	varchar	255	Not null	To store project name of loan
project_report_path	varchar	255	Not null	To store project reports path
questionnaire_score	int	11	Not null	To store questionnaire score
status	enum	255	Not null	To store loan status
user_accepted	bit	1	Not null	To store is the loan accepted by user
questionnaire_id	bigint	11	Not null	To store questionnaire score
user_id	bigint	11	Not null	To store user id
disburse_amount	double	8	Not null	To store disburse amount
disbursed_amount	double	8	Not null	To store disbursed amount
amount_per_month	double	8	Not null	To store how much amount per month is to paid
amount_per_year	double	8	Not null	To store how much amount per year is to paid
questonnaire_id	bigInt	11	Foreign key	To store questionnaire id

user_id	bigint	11	Foreign key	To store user id
---------	--------	----	-------------	------------------

Table name: loan_payment

Purpose: To store the payment details of loan.

Column	Type	Size	Constraints	Description
id	int	11	Primary key	To store loan payment id
created_at	datetime	6	Not Null	To store when loan payment is created
updated_at	datetime	6	Not Null	To store when loan payment is updated
amount_paid	double	8	Not null	To store how much amount is paid
is_on_time	bit	1	Not Null	To store payment is on time
payment_date	datetime	6	Not Null	To store payment date
remarks	varchar	255	Not Null	To store remarks
loan_id	bigint	11	Not Null	To store the id of loan
loan_id	bigint	11	Foreign key	To store the id of loan

Table name: question

Purpose: To store the questions.

Column	Type	Size	Constraints	Description
id	int	11	Primary key	To store question id
created_at	datetime	6	Not Null	To store when loan Question is created
updated_at	datetime	6	Not Null	To store when loan Question is updated
question_text	varchar	255	Not null	To store question text
question_type	enum	255	Not null	To store question type
questionuuid	varchar	255	Not null	To store question uuid
questionnaire_id	bigint	11	Not null	To store questionnare id
questionnaire_id	bigint	11	Foreign key	To store questionnare id

Table name: questionnaire

Purpose: To store the questionnaire.

Column	Type	Size	Constraints	Description
id	int	11	Primary key	To store questionnaire id
created_at	datetime	6	Not Null	To store when questionnaire is created
updated_at	datetime	6	Not Null	To store when questionnaire is updated
description	varchar	255	Not null	To store description
form_url_id	varchar	255	Not null	To store form url id
loan_category	tinyint	1	Not null	To store loan category
questionnaire_type	tinyint	1	Not null	To store questionnaire type
title	varchar	255	Not Null	To store title

Table name: rule

Purpose: To store the rules for loan.

Column	Type	Size	Constraints	Description
id	int	11	Primary key	To store rule id
type	varchar	512	Not null	To store rule type
nace	varchar	255	Not null	To store rule nace
sector	varchar	255	Not null	To store rule sector
activity_number	varchar	255	Not null	To store activity number
activity	varchar	255	Not null	To store activity
contribution_type	text	255	Not null	To store contribution type
description	text	255	Not null	To store description
substantial_contribution_criteria	text	255	Not null	To store substantial contribution criteria
climate_mitigation_dnsh	text	255	Not null	To store climate mitigation dnsh
circular_economy_dnsh	text	255	Not null	To store circular economy dnsh
climate_adaptation_dnsh	text	255	Not null	To store climate adaptaton dnsh
water_dnsh	text	255	Not null	To store water dnsh
pollution_prevention_dnsh	text	255	Not null	To store pollution prevention dnsh
biodiversity_dnsh	text	255	Not null	To store iodiversity dnsh
footnotes	text	255	Not null	To store footnotes

Table name: user

Purpose: To store the user details.

Column	Type	Size	Constraints	Description
id	int	11	Primary key	To store user id
created_at	datetime	6	Not Null	To store when user is created
updated_at	datetime	6	Not Null	To store when user is updated
address	varchar	255	Not Null	To store user address
email	varchar	255	Not Null	To store email
full_name	varchar	255	Not Null	To store user full name
organization	varchar	255	Not Null	To store which organization
password	varchar	255	Not Null	To store password
phone	varchar	255	Not Null	To store phone number
role	varchar	255	Not Null	To store user role
sub_role	varchar	255	Not Null	To store user sub role

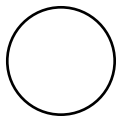
4.1.1 DATA FLOW DIAGRAM(DFD)

The Data Flow Diagram (DFD) represents a system at any level of detail with a graphic network of symbols showing data flows, data stores, data processes, and data sources. The purpose of DFD is to provide a semantic bridge between users and system developers. The diagram is the basis of structured system analysis. A level 0 DFD, also called a fundamental system model or a context model represents the entire software elements as a single bubble with input and output indicated by incoming and outgoing arrows respectively. Additional process and information flow parts are represented in the next level i.e., Level 1 DFD. Each of the processes represented at Level 1 are sub functions of the overall system depicted in the context model. Any processes which are complex in Level 1, will be further represented into sub functions in the next level, i.e., in level 2. Data flow diagrams illustrate how data is processed by a system in terms of inputs, and output represented by major components or functions with Circles.

Components of Data Flow Diagrams

There are only four symbols that are used in the drawing of data flow diagrams. These are explained below together with the rules that apply to them.

- Process: A process that represents some amount of work being performed on data.



Circle or Bubble

- External Entity: This represents any outside agency, which interact with the system. It represents the source or destination of data for the system under consideration.



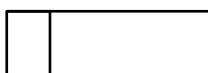
Rectangle

- Data Flow: The data flow portrays an interface among different components in a DFD. It represents flow of data between a process and an external entity or between a process and data store.



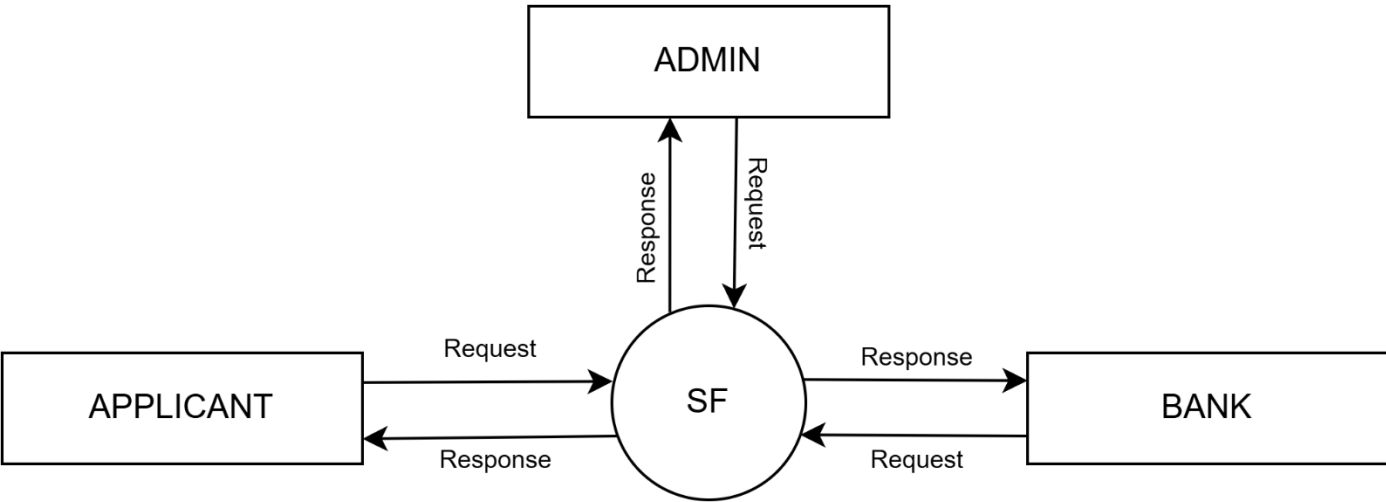
Arrow

- Data Stores: A data store is a place for holding information within the system.

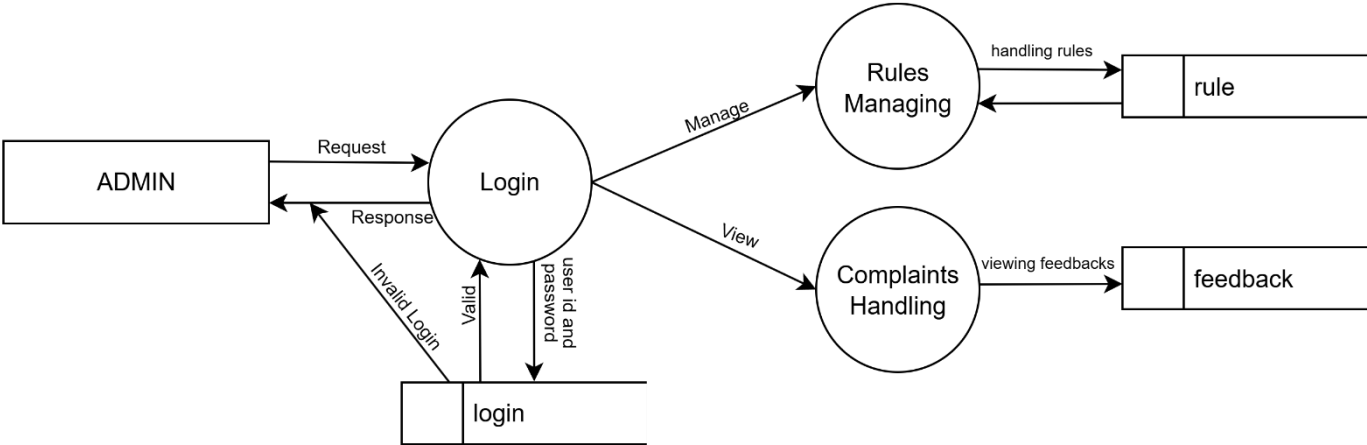


One-end opened rectangle

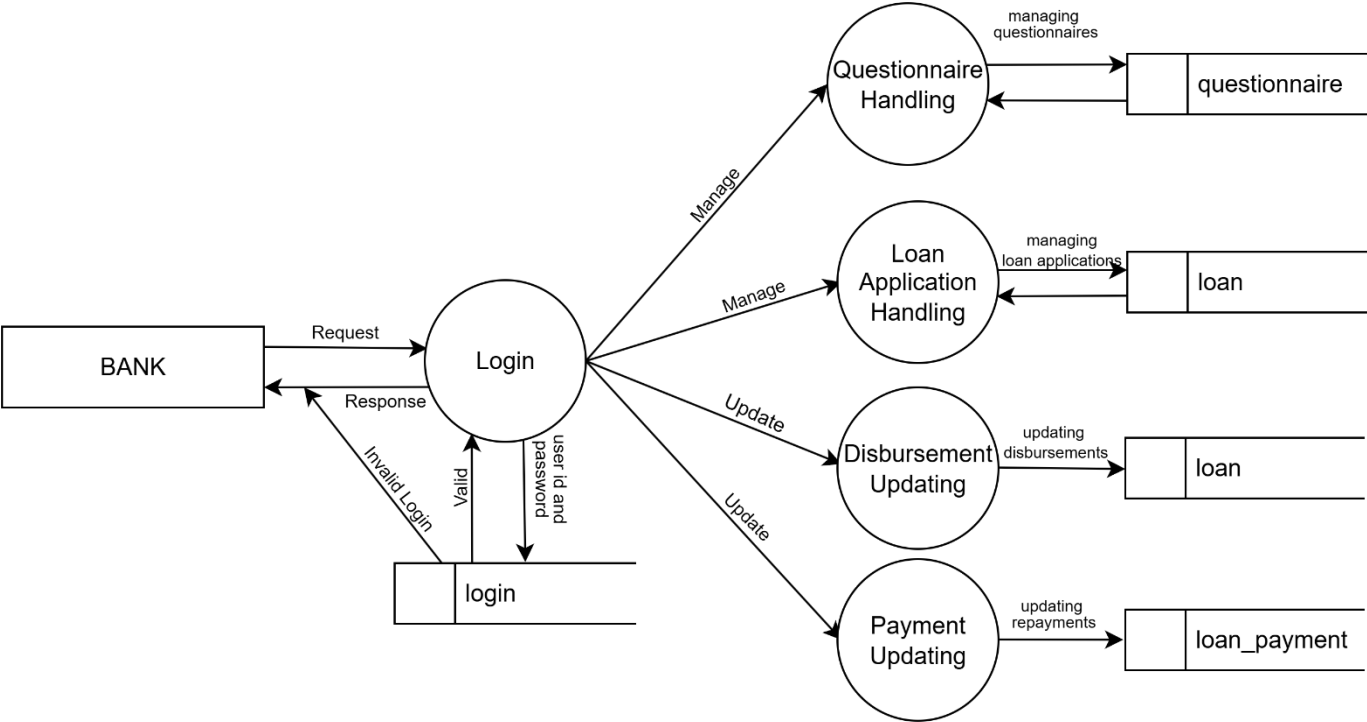
Context level



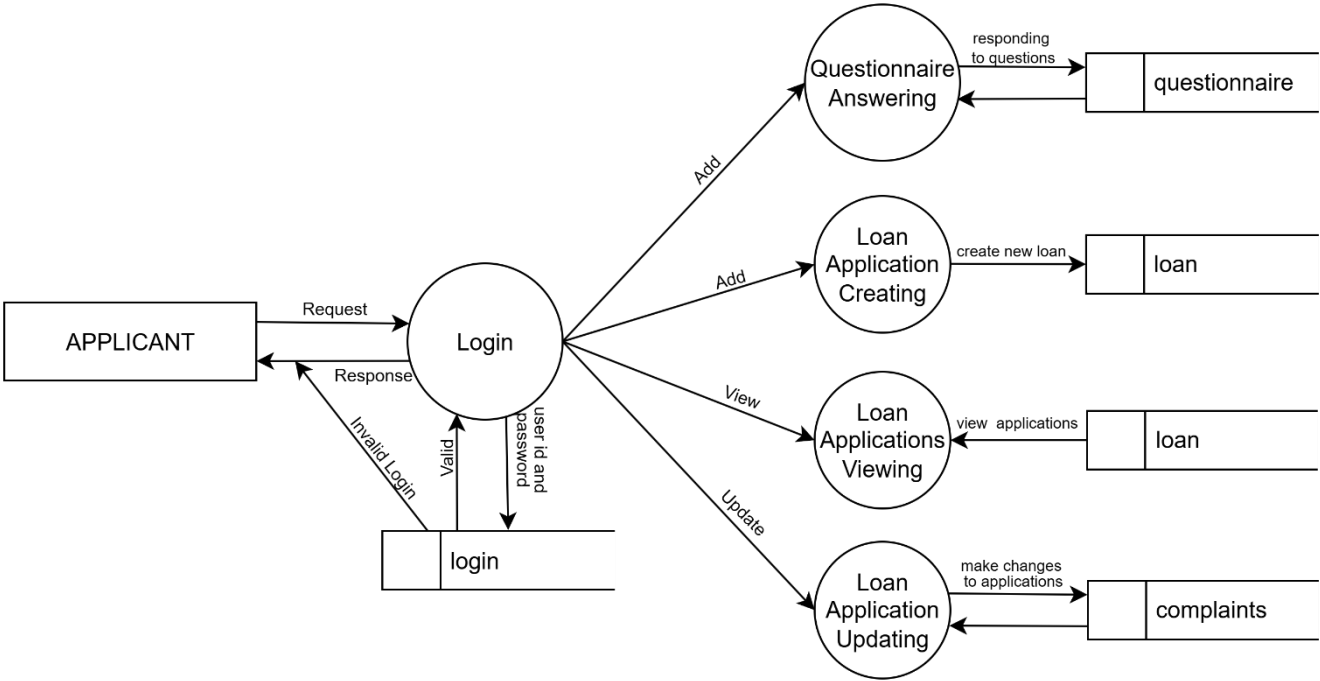
Level 1 Admin



Level 1 Bank



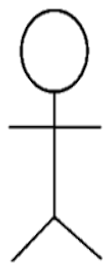
Level 1 Applicant



4.1.2 USE CASE DIAGRAM

A use case is a set of scenarios that describes the interaction between user and system. A use case diagram displays the relationship among the actors and use cases. The two main components of a use case diagram are cases and actors. A use case defines the interactions between external actors and system under consideration to accomplish goals. Actors must be able to make decisions, but need not to be human: An actor must be a person, a company or an organization.

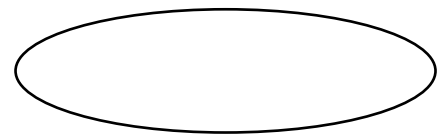
Components of Use Case Diagram



Actor



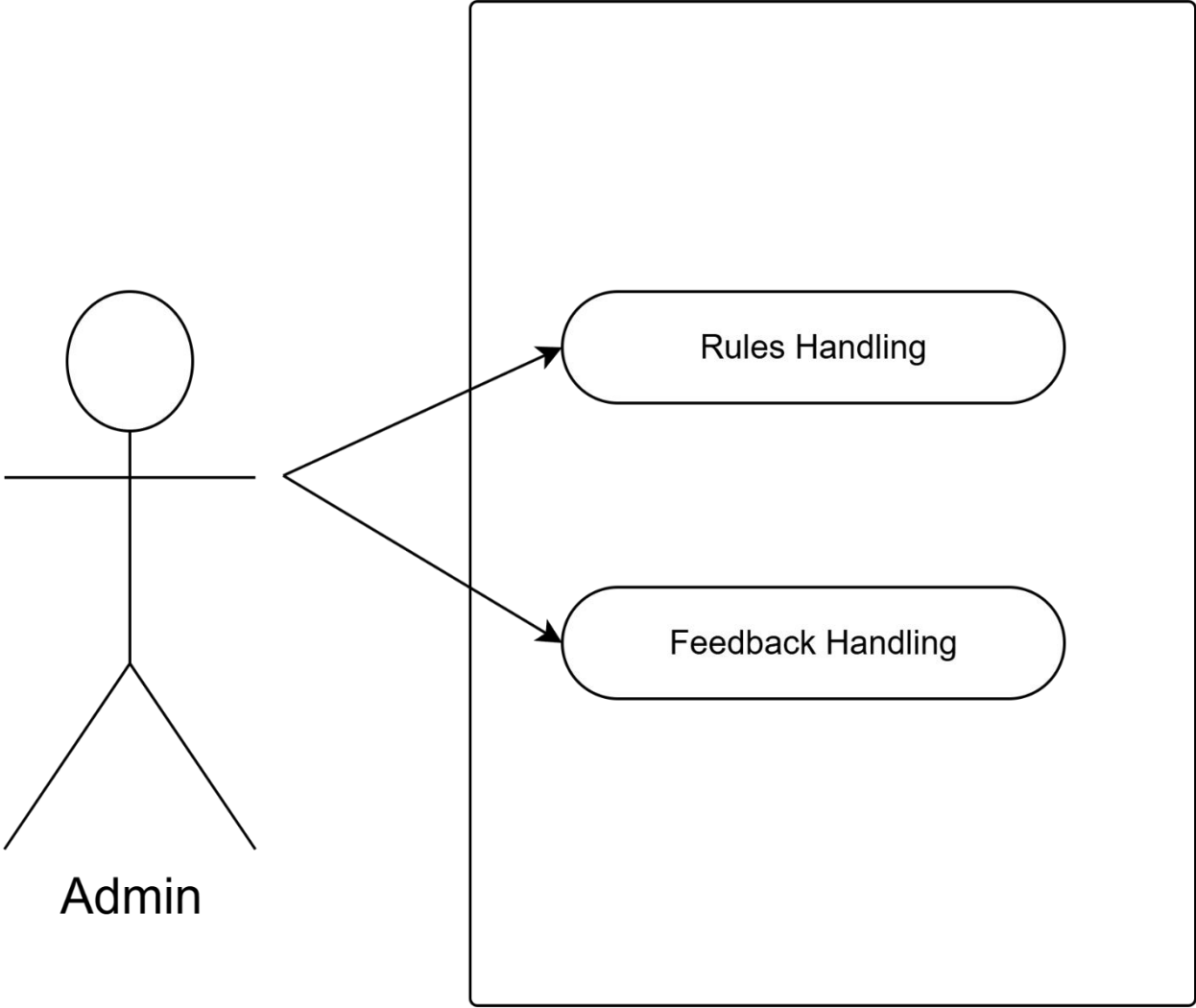
System Boundary



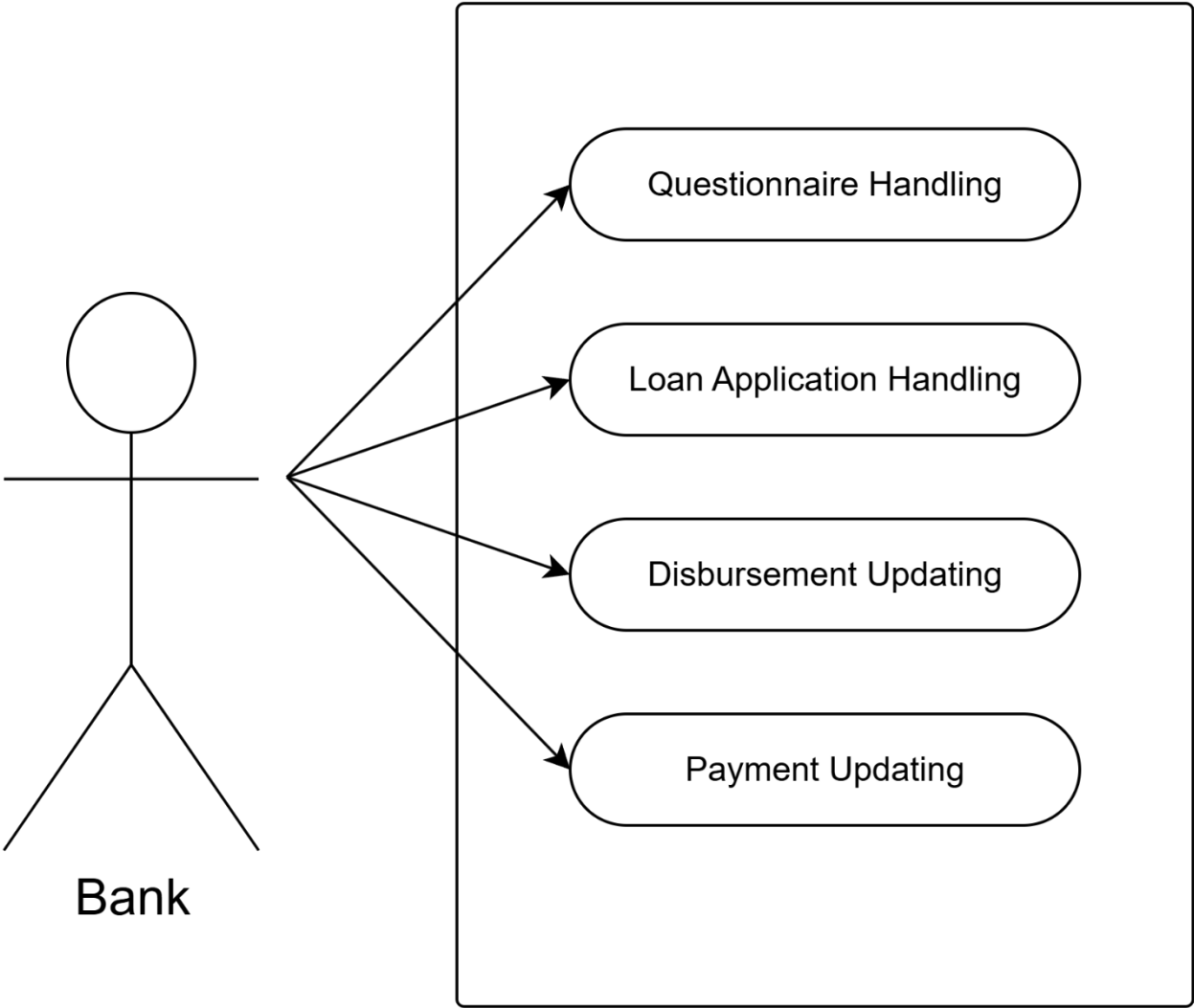
Use Case

A use case represents a user or another system that will interact with the system you are modeling. A use case is an external view of the system that represents actions that user might perform to complete a task. Use cases are used in almost every project. They are helpful in exposing requirements and planning the project. During the initial stage of a project most use case should be defined, but as the project continues might become visible.

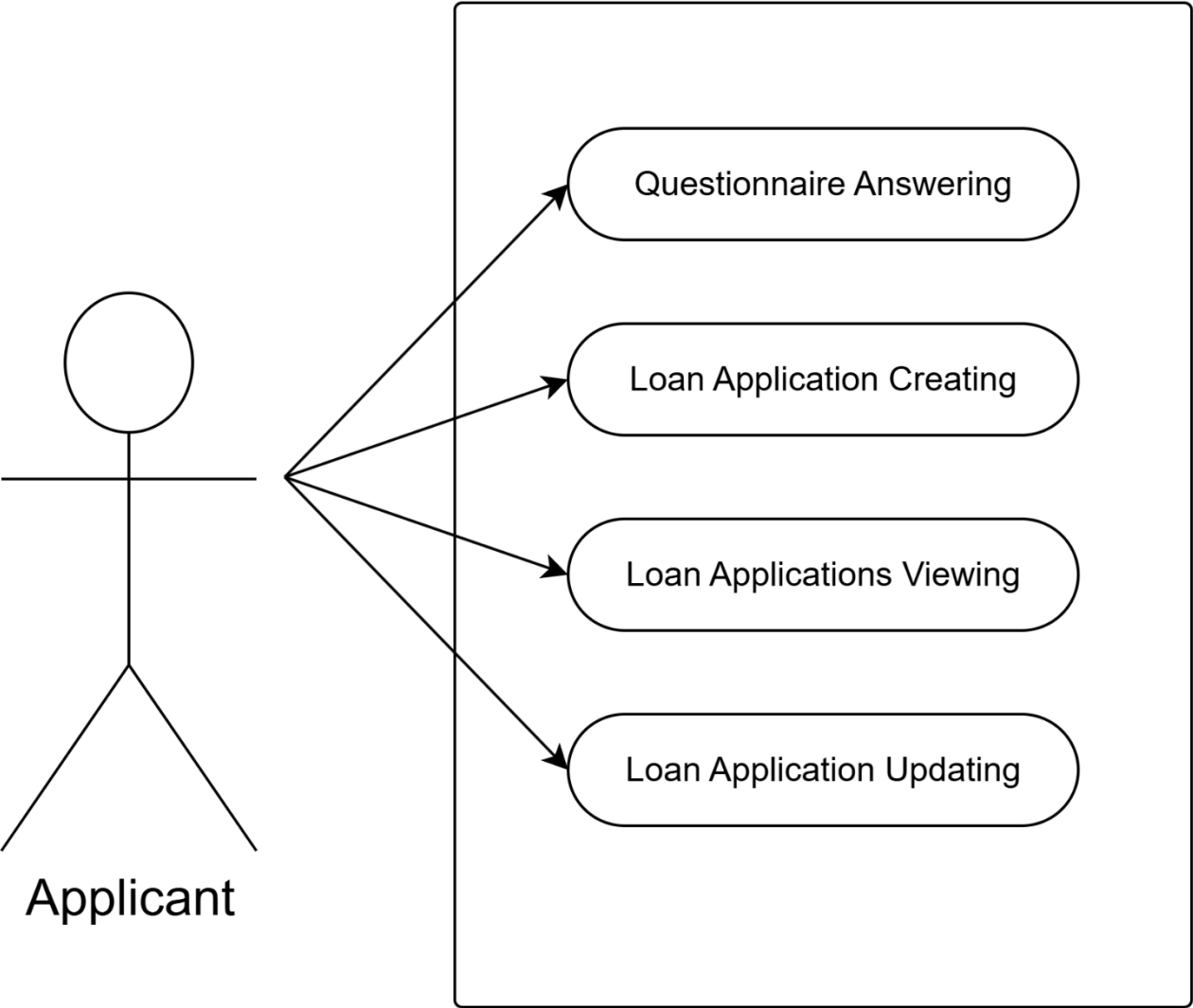
Use case 1: Admin



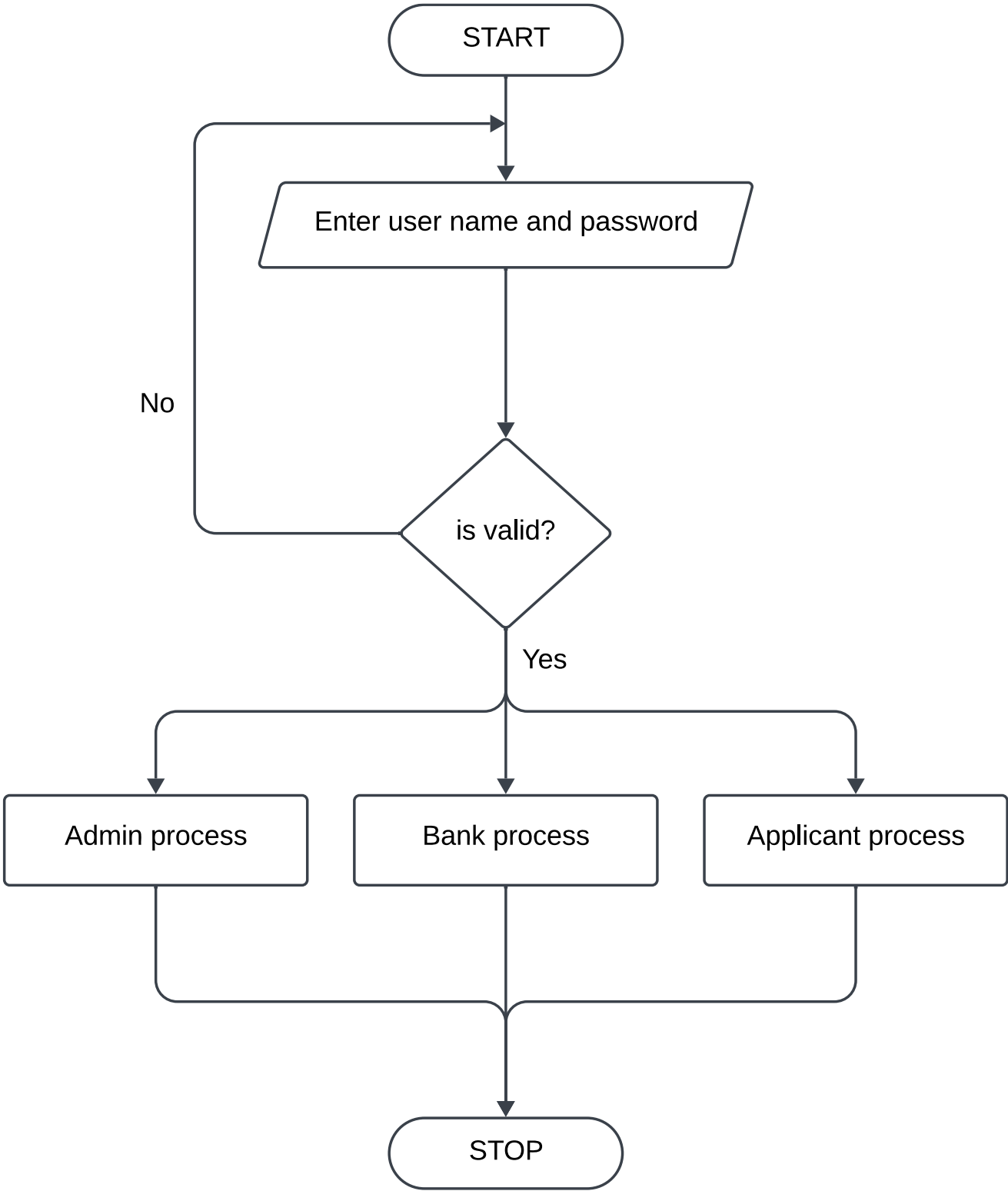
Use case 2: Bank



Use case 3: Applicant



4.1.3 FLOWCHART



4.2 PROCEDURAL DESIGN

Procedural design is best used to model programs that have an obvious flow of data from input to output. It represents the architecture of a program as a set of interacting processes that data pass data from one another. The steps involved in this are:

- Computer procedure: Specify what functions will be carried out on computer, what will be different programs and in what sequence program will be run.
- Non-computer procedure: Specify the manual procedures for feeding input data, receiving outputs etc.

4.3 INTERFACE DESIGN

The purpose of interface design is to communicate effectively through form designs: there are several major requirements:

- The form title must clearly identify its purpose. Columns and rows should be labeled to avoid confusion. The form should be identified by firm name or code number to make it easy to work with.
- The form must be easy to use and fill out. It should be legible, intelligible and uncomplicated.
- The data requested should reflect a logical sequence.

4.4 INPUT DESIGN

The input design is the process of converting the user-oriented inputs into the computer-based format. The goal of designing input data is to make automation as easy and free from errors as possible. The input design requirements such as user friendliness, consistent format and interactive dialogue for giving the right message and help for the user at right time are also considered for the development of the project.

The following points should be considered while designing the input:

- What data to input?
- What medium to use?
- How the data should be arranged or coded?

Inaccurate input data is the most common cause of error in processing data. The arrangement of messages as well as placement of data, headings and titles on display screens or source document is also a part of input design. The design of input also includes specifying the means by which end user and system operators direct the system what action to take.

4.5 OUTPUT DESIGN

Output generally refers to the results and information that are generated by the system. When output, system analyst must accomplish the following:

- Determine what information to present.
- Decide whether to display, print the information and select the output medium.
- Arrange the presentation of information in an acceptable format.
- Decide how to distribute the output to intended recipients.

SYSTEM CODING

5. SYSTEM CODING

System Coding is the process of converting the system design into executable programs using a suitable programming language. It involves writing clean, structured, and efficient source code that implements the functional and non-functional requirements of the system. The coding step transforms design specifications into a working system, ensuring that logic, rules, and workflows are translated into machine-executable instructions. Quality of coding depends on consistent style, readability, proper documentation, and adherence to coding standards, which together reduce errors and improve maintainability.

During this phase, the backend is developed using Java Spring Boot to build APIs and handle business logic, while the frontend is implemented with ReactJS for a responsive interface. Database operations are managed using MySQL with the help of Hibernate and JPA for ORM. Build tools like Maven are used for project management, and GitHub ensures version control and collaboration.

CODE

User.java

```
package com.Detroit.detroit.sf.entity;

import com.Detroit.detroit.enums.Role;
import com.Detroit.detroit.enums.SubRole;
import com.Detroit.detroit.library.EntityDetails;
import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class User extends EntityDetails {

    private String fullName;
    @Column(unique = true, nullable = false)
    private String email;
```

```
@Column(nullable = false)
```

```
private String password;
```

```
@Enumerated(EnumType.STRING)
```

```
@Column(nullable = false)
```

```
private Role role;
```

```
private SubRole subRole;
```

```
@Column(unique = true, nullable = false)
```

```
private String phone;
```

```
private String address;
```

```
private String organization;
```

```
}
```

```
package com.Detroit.detroit.enums;
```

```
public enum Role {
```

```
    ADMIN,
```

```
    BANK,
```

```
    APPLICANT
```

```
}
```

```
package com.Detroit.detroit.enums;
```

```
public enum SubRole {
```

```
    ENTERPRISE, // Applicant
```

```
    GOVERNMENT, // Applicant
```

```
}
```

UserRepository.java

```
package com.Detroit.detroit.sf.repository;
```

```
import com.Detroit.detroit.sf.entity.User;
```

```
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import org.springframework.stereotype.Repository;
```

```
import java.util.Optional;
```

@Repository

```
public interface UserRepository extends JpaRepository<User, Long> {  
    Optional<User> findByEmail(String email);  
}
```

UserService.java

```
package com.Detroit.detroit.sf.service;
```

```
import com.Detroit.detroit.dto.Login;  
import com.Detroit.detroit.sf.entity.User;  
import com.Detroit.detroit.sf.repository.UserRepository;  
import jakarta.persistence.EntityNotFoundException;  
import lombok.AllArgsConstructor;  
import org.springframework.security.crypto.password.PasswordEncoder;  
import org.springframework.stereotype.Service;
```

```
import java.util.List;
```

@Service

@AllArgsConstructor

```
public class UserService {
```

```
    public final UserRepository userRepository;  
    public PasswordEncoder passwordEncoder;
```

```
//get all user
```

```
public List<User> getAllUser(){  
    return userRepository.findAll();  
}
```

```
//get Specific user by id
```

```
public User getUserById(Long id) {  
    return userRepository.findById(id).orElseThrow(()-> new EntityNotFoundException("User  
not found At id:"+id));  
}
```

```
//Create a new user
```

```
public User addUser(User user) {
```

```
        user.setPassword(passwordEncoder.encode(user.getPassword()));
        return userRepository.save(user);
    }

    // Get user by email and verify password
    public User getUserByEmail(Login login) throws EntityNotFoundException {

        User findUser = userRepository.findByEmail(login.getEmail())
            .orElseThrow(() -> new EntityNotFoundException("User not found with email: " +
login.getEmail()));
        if(login.getEncrypted()) {
            if(login.getPassword().equals(findUser.getPassword()) &&
login.getRole().equals(findUser.getRole())) {
                return findUser;
            }
            throw new IllegalArgumentException("Wrong credentials");
        }
        else if (passwordEncoder.matches(login.getPassword(), findUser.getPassword())) {
            return findUser;
        }
        throw new IllegalArgumentException("Wrong credentials");
    }

    //update password
    public User updatePassword(String email, String password){
        User user = userRepository.findByEmail(email)
            .orElseThrow(() -> new EntityNotFoundException("User not found with email: " +
email));
        user.setPassword(passwordEncoder.encode(password));
        return userRepository.save(user);
    }

    // Update an existing user
    public User updateUser(Long id, User updateUser){
        return userRepository.findById(id).map( user -> {
            user.setFullName(updateUser.getFullName());
        });
    }
```

```

        user.setEmail(updateUser.getEmail());
        user.setPhone(updateUser.getPhone());
        user.setAddress(updateUser.getAddress());
        user.setOrganization(updateUser.getOrganization());
        user.setRole(updateUser.getRole());
        return userRepository.save(user);
    }).orElseThrow(() -> new IllegalArgumentException("User not found with id:"+id));
}
//delete an user
public void deleteUser(Long id){
    if(!userRepository.existsById(id)){
        throw new EntityNotFoundException();
    }
    userRepository.deleteById(id);
}
}

```

UserController.java

```

package com.Detroit.detroit.sf.controller;

import com.Detroit.detroit.dto.Login;
import com.Detroit.detroit.sf.entity.User;
import com.Detroit.detroit.sf.service.UserService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("api/user")
public class UserController {

    public final UserService userService;

    public UserController(UserService userService){
        this.userService=userService;
    }

    //get all users
    @GetMapping
    public ResponseEntity<List<User>> getAllUsers() {
        return ResponseEntity.ok(userService.getAllUser());
    }
}

```

```

//get a user by id
@GetMapping("/{id}")
public ResponseEntity<User> getUserById(@PathVariable Long id) {
    return ResponseEntity.ok(userService.getUserById(id));
}

//Create a new user
@PostMapping
public ResponseEntity<User> createUser(@RequestBody User user){
    return ResponseEntity.ok(userService.addUser(user));
}

// Log in user
@PostMapping("/login")
public ResponseEntity<User> loginUser(@RequestBody Login loginCred) {
    return ResponseEntity.ok(userService.getUserByEmail(loginCred));
}

//update a user
@PutMapping("/{id}")
public ResponseEntity<User> updateUser(@PathVariable Long id,@RequestBody User
user){
    return ResponseEntity.ok(userService.updateUser(id, user));
}

// Delete a user
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteUser(@PathVariable Long id){
    userService.deleteUser(id);
    return ResponseEntity.noContent().build();
}
}

```

Loginpage.jsx

```

import React, { useState } from 'react';
import axios from 'axios';
import './Loginpage.css';
import {
    FaUser, FaEnvelope, FaMapMarkerAlt, FaBuilding, FaPhone,
    FaEye, FaEyeSlash, FaBriefcase
} from "react-icons/fa";
import { saveUser } from '../hooks/LocalStorageUser';
import { useNavigate } from "react-router";
import API from '../hooks/API';

const useApi = API();

const Loginpage = ({ user }) => {

    const navigate = useNavigate();

    const [showPass, setShowPass] = useState(false);

```

```

const [isForgotPassword, setIsForgotPassword] = useState(false);
const [isRegistration, setIsRegistration] = useState(false);

const [loginData, setLoginData] = useState({
  email: "",
  password: ""
});

const [forgotEmail, setForgotEmail] = useState("");
const [registerData, setRegisterData] = useState({
  firstName: "",
  lastName: "",
  phone: "",
  email: "",
  password: "",
  address: "",
  organization: "",
  role: "",
  agreed: false
});

const [message, setMessage] = useState("");
const [error, setError] = useState("");

const handleLoginChange = (e) => {
  const { name, value } = e.target;
  setLoginData((prev) => ({
    ...prev,
    [name]: value
  }));
};

const handleLoginSubmit = async (e) => {
  e.preventDefault();
  const role = window.location.pathname.split('/')[2].toUpperCase();
  const data = {
    email: loginData.email,
    password: loginData.password,
    role: role
  };

  try {
    const response = await axios.post(useApi.url+'/user/login', data, {
      headers: { 'Content-Type': 'application/json' }
    });
    console.log('Login success:', response.data);
    saveUser(response.data);
    navigate("/");
  } catch (error) {
    alert("Error occurred during login");
    console.error('Login error:', error.response?.data || error.message);
  }
};

```

```

const handleForgotEmailChange = (e) => setForgotEmail(e.target.value);

const handleForgotPasswordSubmit = async (e) => {
  e.preventDefault();
  try {
    const response = await axios.post(useApi.url+'/user/forgot-password', { email: forgotEmail
  });
  setMessage('A password reset link has been sent to your email.');
```

```

  setError("");
  } catch (error) {
    alert("Error occurred");
    setError('An error occurred, please try again.');
```

```

    setMessage("");
  }
};

const handleRegisterChange = (e) => {
  const { name, value, type, checked } = e.target;
  setRegisterData((prev) => ({
    ...prev,
    [name]: type === 'checkbox' ? checked : value
  }));
};

const handleRegisterSubmit = async (e) => {
  e.preventDefault();
  const data = {
    fullName: `${registerData.firstName} ${registerData.lastName}`,
    phone: registerData.phone,
    email: registerData.email,
    password: registerData.password,
    address: registerData.address,
    role: user === "APPLICANT" ? user : registerData.role,
    organization: registerData.organization,
    subRole: user === "APPLICANT" ? registerData.role : null
  };

  try {
    const response = await axios.post(useApi.url+'/user', data, {
      headers: { 'Content-Type': 'application/json' }
    });
    console.log('Registration success:', response.data);
    saveUser(response.data);
    navigate('/');
  } catch (error) {
    alert("Error occurred during registering");
    console.error('Registration error:', error.response?.data || error.message);
  }
};

const registerLink = () => {
  setIsRegistration(true);
  setIsForgotPassword(false);
};

```

```

const loginLink = () => {
  setIsRegistration(false);
  setIsForgotPassword(false);
};

return (
  <div className="container-login">
    <div style={{
      width: '100%',
      maxWidth: isRegistration ? '500px' : '400px'
    }} className="wrapper">
      {/* Login Form */}
      {!isForgotPassword && !isRegistration && (
        <div style={{width: "100%"}} className="form-box login">
          <form onSubmit={handleLoginSubmit}>
            <h1>Login</h1>
            <div className="input-box">
              <input
                type="text"
                name="email"
                placeholder="Email"
                required
                value={loginData.email}
                onChange={handleLoginChange}
              />
              <FaUser className="icon" />
            </div>

            <div className="input-box">
              <input
                type={showPass ? "text" : "password"}
                name="password"
                placeholder="Password"
                required
                value={loginData.password}
                onChange={handleLoginChange}
              />
              <div className="icon" onClick={() => setShowPass(!showPass)} style={{ zIndex:
100, cursor: 'pointer' }}>
                {showPass ? <FaEye /> : <FaEyeSlash />}
              </div>
            </div>

            <button type="submit">Login</button>
            <div className="register-link">
              <p>Don't have an account? <a style={{ cursor: 'pointer' }}
onClick={registerLink}>Register</a></p>
            </div>
          </form>
        </div>
      )}
    </div>
  )

```

```

{/* Registration Form */}
{isRegistration && (
  <div className="form-box register">
    <form onSubmit={handleRegisterSubmit}>
      <h1>Register</h1>
      <div className="name-row">
        <div className="input-box half">
          <input
            type="text"
            name="firstName"
            placeholder="First Name"
            required
            value={registerData.firstName}
            onChange={handleRegisterChange}
          />
          <FaUser className="icon" />
        </div>
        <div className="input-box half">
          <input
            type="text"
            name="lastName"
            placeholder="Last Name"
            required
            value={registerData.lastName}
            onChange={handleRegisterChange}
          />
          <FaUser className="icon" />
        </div>
      </div>

      <div className="input-box">
        <input
          type="text"
          name="phone"
          placeholder="Phone"
          required
          value={registerData.phone}
          onChange={handleRegisterChange}
        />
        <FaPhone className="icon" />
      </div>

      <div className="input-box">
        <input
          type="email"
          name="email"
          placeholder="Email"
          required
          value={registerData.email}
          onChange={handleRegisterChange}
        />
        <FaEnvelope className="icon" />
      </div>

```

```

<div className="input-box">
  <input
    type={showPass ? "text" : "password"}
    name="password"
    placeholder="Password"
    required
    value={registerData.password}
    onChange={handleRegisterChange}
  />
  <div onClick={() => setShowPass(!showPass)} style={{ cursor: 'pointer' }}>
    {showPass ? <FaEye className="icon" /> : <FaEyeSlash className="icon" />}
  </div>
</div>

{user === "APPLICANT" && (
  <
    <div className="input-box">
      <input
        type="text"
        name="address"
        placeholder="Address"
        required
        value={registerData.address}
        onChange={handleRegisterChange}
      />
      <FaMapMarkerAlt className="icon" />
    </div>

    <div className="input-box">
      <input
        type="text"
        name="organization"
        placeholder="Organization"
        value={registerData.organization}
        onChange={handleRegisterChange}
      />
      <FaBuilding className="icon" />
    </div>

    <div className="input-box">
      <select
        name="role"
        value={registerData.subRole}
        onChange={handleRegisterChange}
        required
      >
        <option value="">Select Role</option>
        <option value="ENTERPRISE">Enterprise</option>
        <option value="GOVERNMENT">Government</option>
      </select>
      <FaBriefcase className="icon" />
    </div>
  </>
)}

```

```
{user === "BANK" && (
  <div className="input-box">
    <select
      name="role"
      value={registerData.role}
      onChange={handleRegisterChange}
      required
    >
      <option value="">Select Role</option>
      <option value="BANK">Bank</option>
      <option value="ADMIN">Admin</option>
    </select>
    <FaBriefcase className="icon" />
  </div>
)}

<button type="submit">Register</button>
<div className="register-link">
  <p>Already have an account? <a style={{ cursor: 'pointer' }}
onClick={loginLink}>Login</a></p>
</div>
</form>
</div>
)}
</div>
</div>
);
};

export default Loginpage;
```

SYSTEM TESTING

6. SYSTEM TESTING

6.1 TESTING

Testing is the last stage of the software development before we release the product to the customer. Software testing is a critical element of software quality assurance and represents the ultimate review of specification, design and coding. Software testing can be looked upon as one among the many processes. Testing cannot show the absence of defects, it can only show that software defects are present.

Software Testing Techniques

The importance of testing and its impact on software cannot be underestimated. The greater visibility of software systems and the cost associated with the software failure are motivating factors for planning through testing. It is not uncommon for a software organization to spend 40% of its effort on testing.

A number of rules that act as Testing Objectives are:

- Testing is the process of executing a program with the aim of finding errors.
- A good test case will have a good chance to find an undiscovered error.
- A successful test case uncovers a new error.

6.2 UNIT TESTING

The first level testing is unit testing. Unit testing concentrates on each unit if the software is implemented in source code. Initially tests focus on each module individually, ensuring that it functions properly as a unit. The modules must then be assembled or integrated to form the complete software package.

There are tests that occur as part of unit testing. The module interfaces are tested to ensure that information properly flows into and out of a program under test. The data structures are also tested for integrity. Boundary conditions are tested to ensure that the module operates properly at boundaries established to limit or restrict processing.

6.3 INTEGRATION TESTING

The next level of testing is often called Integration testing in which many tested modules are combined into a sub-system, which are then tested. Data can be lost across an interface; one module can have an adverse effect on the other sub functions, when combined may not produce the desired major functions. Integrated testing is the systematic testing for constructing the uncovered errors within the interface. This testing was done with sample data. The developed system has run successfully for this sample data. The need for integrated test is to find the overall system performance

Data Validation Testing

Data Validation is the process of testing the accuracy of data; a set of rules you can apply to a control to specify the type and range of data that users can enter. It can be used to display error alerts when users enter incorrect values into a form. Rather than checking for errors after a form is completed; data validation verifies values as the form is being filled out.

A strategy for software testing integrates software test case design method into a well-planned series of steps that result in the successful construction of the software. The strategy provides a road map that describes the step to be conducted as part of testing, when these steps are planned and then undertaken, and how much effort, time and resources will be required. Therefore, any testing strategy must incorporate test planning, test case, design, test execution and resultant data collection and evaluation. A software testing strategy should be flexible enough to promote a customized testing approach.

System tests are carried out to validate a duly developed system with a view assuring that it meets its requirements. There are essentially three kinds of system testing.

1. Alpha Testing

It refers to the system testing that is carried out by the test team within the organization.

2. Beta Testing

Beta testing is the system testing performed by a selected group of friendly customers.

3. Acceptance Testing

Acceptance testing is the system testing performed by the customer to determine whether or not to accept the delivery of the system. The application is tested to ensure the requirements. Different sets of input data are entered to validate the system. In all cases the system produces a reasonable output.

6.3 SYSTEM TESTING

System testing is a type of software testing that evaluates the complete and integrated software system to ensure it meets the specified requirements. It is performed after integration testing and checks the system as a whole, including all modules, interfaces, and interactions with external systems. The goal is to verify that the software behaves correctly in its intended environment, covering both functional and non-functional aspects like performance, security, and usability.

6.4 USER ACCEPTANCE TESTING

Acceptance testing is a key factor to the success of any system. The system under the consideration was treated for user acceptance by constantly keeping in touch with the prospective system user at the time of developing and making changes wherever and whenever required.

White Box Testing

White box testing strategy deals with the internal logic and structure of the code. White box testing is also called glass, structural, open box or clear box testing. The tests written based on the white box testing strategy incorporate coverage of the code written, branches, paths, statements and internal logic of the code etc.

In order to implement white box testing, the tester has to deal with the code and hence is needed to possess knowledge of coding and logic i.e., internal working of the code. The White box test also needs the tester to look into the code and find out which unit/ statement/ chunk of the code is malfunctioning.

Black Box Testing

Black Box Testing is not a type of testing; it instead is a testing strategy, which does not need any knowledge of internal design or code etc. As the name “black box” suggests, no knowledge of internal logic or code structure is required. The types of testing under this strategy are totally based / focused on the testing for requirements and functionality of the work product/software application. Black box testing is sometimes also called “Opaque testing”, “Functional/Behavioral Testing” and “Closed Box Testing”.

Test Cases

A test case in software engineering is a set of conditions or variables under which a tester will determine whether an application or software system is working correctly or not. The mechanism for determining whether a software system or system has passed or failed such a test is known as test oracle. In some settings, an oracle could be a requirement or use case, while in other side could be a heuristic.

Need For Testing

Testing is essential as

- The existence of program defects or inadequacies is inferred.
- Test the performance of the system.
- Verifies whether the software behaves as intended by its designer.

SYSTEM IMPLEMENTATION AND MAINTENANCE

7. SYSTEM IMPLEMENTATION AND MAINTENANCE

Systems implementation is the process of: defining how the information system should be built (i.e., physical system design), ensuring that the information system is operational and used, ensuring that the information system meets quality standard (i.e., quality assurance)

System maintenance is an umbrella term that encompasses various forms of computer maintenance needed to keep a system running. The two main components of system maintenance are preventive and corrective maintenance.

7.1 SYSTEM IMPLEMENTATION

System implementation phase is the most difficult in the lifecycle. It is defined as the process of converting a new or revised system design into an operational one. Implementation includes all those activities that take place to convert from the old system to new. The old systems consist of manual operations, which are operated in very different manner from the proposed new system. These are several methods for handling the implementation and the consequent conversion from the old system to the new computerized system.

The most secure method for conversion from the old system to the new system is to run both systems in parallel. In this approach a person may operate in the manual older processing system as well as start operating the computerized system, we can depend upon the manual system.

Another commonly used method is a direct cut over from the existing manual system to the computerized system. The system may work within a week or a day. There are no parallel activities. However, there is no remedy in case of a problem. This strategy requires careful planning. A working version of the system can also be implemented in one part of the organization and the personal will be piloting the system and changes can be made as and when required. But this method is less preferable due to the loss of the entire system. This type of conversion is usually difficult and is not properly planned; it can give way to a number of problems. Implementations of a modified application to replace an existing one, using the same computer are the next method. This type of conversion is relatively easy to handle. Provided there are no changes in the file. In our project we implemented a new computer system to replace an existing one.

7.2 SYSTEM MAINTENANCE

Maintenance is a characteristic of design and implementation, which is expressed, as the probability that an item will be retained in or restored to a specific condition within a given period of time, when maintenance is performed to accordance with the prescribed procedures and resource.

Maintenance is the enigma of system development. It holds the software industry captive, tying up programming resources. Analyst and programmers spend far more time maintaining program than they do writing them.

Maintenance can be classified as corrective, adaptive or prefecture. Corrective maintenance means repairing processing or performance failures or making changes because of previously uncorrected problem or false assumptions. Adaptive maintenance means repairing processing or performance or modifying the program to respond to the users additional or changing needs. Of this type more time and money are spent on prefecture than on corrective and adaptive maintenance.

Technical and management approaches to the maintenance phase can be implemented with little upheaval. However, tasks performed during the software engineering process define maintainability and have an important on the success of any maintenance approach.

7.3 FUTURE SCOPE & FUTURE ENHANCEMENT

The future scope of the **Sustainable Finance** lies in its potential to become a comprehensive sustainability-focused evaluation and loan management platform. As awareness and regulatory emphasis on green financing continue to grow, the system can be expanded to support additional sustainability domains, financial instruments, and cross-institutional collaboration.

One major direction for future development is integrating real-time ESG (Environmental, Social, and Governance) data sources to enhance assessment accuracy. The system could also incorporate project lifecycle tracking, enabling banks to monitor the actual sustainability performance of approved projects beyond the initial evaluation.

Another key enhancement is the integration with national and international sustainability taxonomies, such as those defined by regulatory bodies or development banks, to ensure global compliance and standardization. Additionally, support for multilingual interfaces and regional regulatory customization would make the system adaptable for banks across different geographies.

From a technical standpoint, the platform can be extended with predictive scoring using pattern-based analysis of past applications, allowing early identification of strong or weak proposals. The document management module could be enhanced to support version control, digital signatures, and AI-based validation of uploaded project reports.

To improve the user experience, future enhancements could include mobile app access for applicants and officers, data visualization dashboards for administrators, and automated alerts on rule changes or disbursement updates.

Security and compliance will remain top priorities. Incorporating advanced encryption, role-based audit trails, and compliance with data privacy laws (such as GDPR or RBI guidelines) will ensure the integrity and trustworthiness of the system. A long-term enhancement may also include blockchain-based logging to add transparency to the scoring and approval process.

Finally, as green finance becomes more collaborative, the system could evolve into a multi-bank platform, enabling shared sustainability criteria and secure inter-bank data exchange — helping to establish a unified green lending ecosystem.

CONCLUSION

8. CONCLUSION

In conclusion, on sustainability criteria. By leveraging structured questionnaires and dynamic scoring mechanisms, the system brings consistency, transparency, and speed to the pre-screening process for green finance. The integration of rule-based decision logic ensures objectivity in determining whether a project aligns with environmental goals such as climate mitigation, water conservation, and biodiversity protection.

The system not only simplifies the evaluation process but also empowers bank administrators to define and update eligibility rules and questionnaires without any technical dependency. Through its modular architecture and role-based interfaces, it offers a seamless experience for applicants, bank officers, and administrators alike. The ability to upload and review project documents further strengthens the decision-making process and supports better documentation and record-keeping.

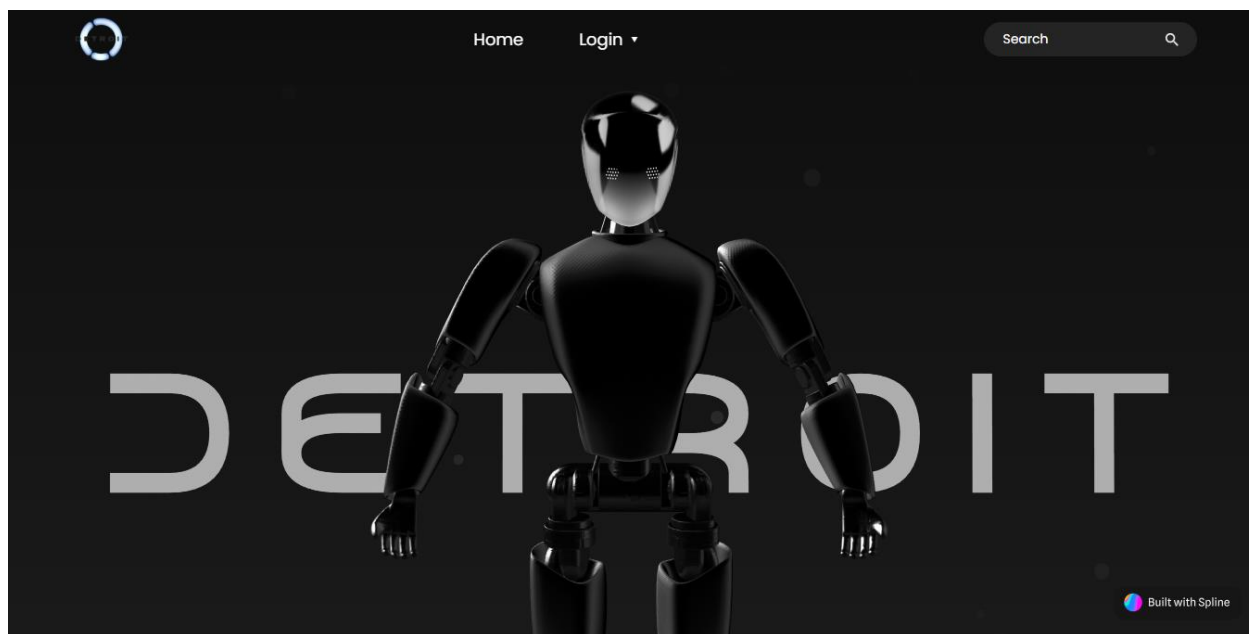
With its scalability and adaptability, the system is well-positioned to support a wide range of sustainable development goals and project types. Its efficient design reduces the manual workload for financial institutions, promotes environmentally conscious lending, and supports long-term sustainability targets. Overall, this platform plays a vital role in advancing green finance practices and offers a forward-looking tool for modern banks seeking to align their operations with sustainable development.

APPENDIX

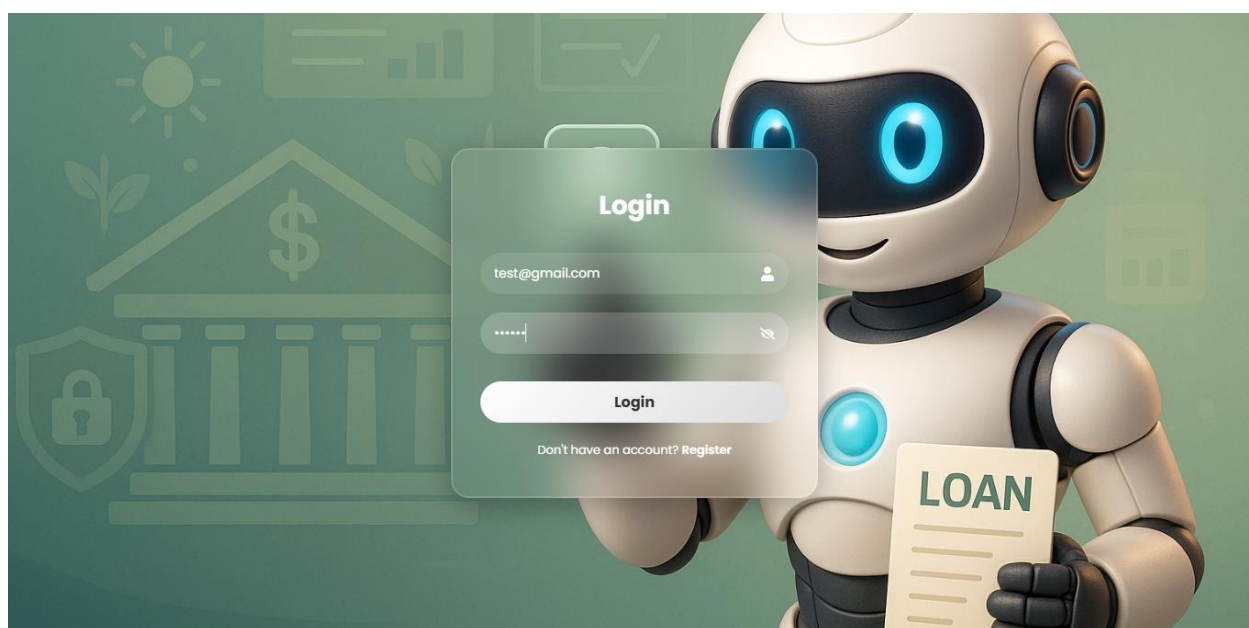
9. APPENDIX

9.3 SCREEN LAYOUTS AND REPORTS

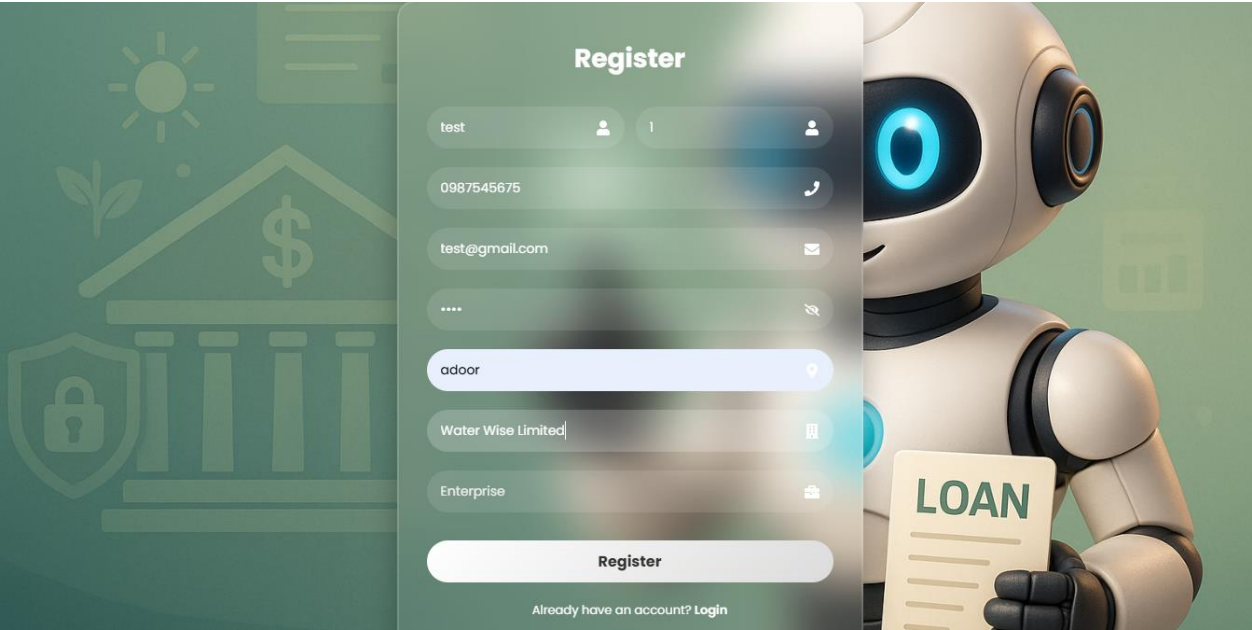
Main homepage



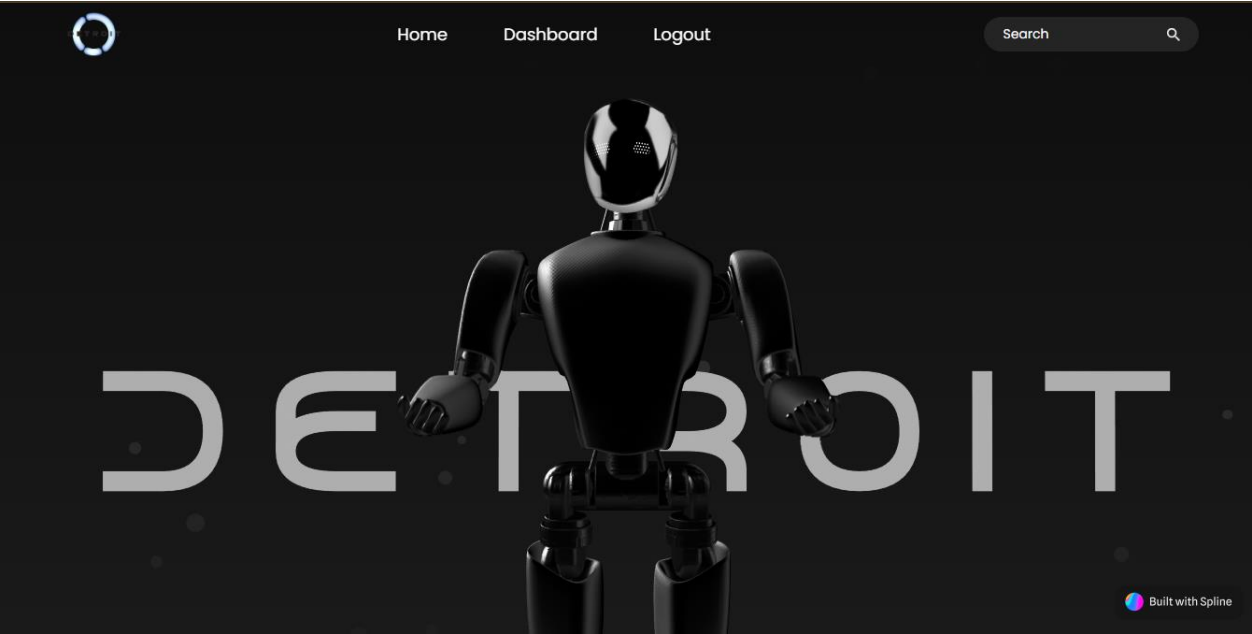
Login page



Registration



Enterprise/Government homepage



Basic Loan Process



HomeDashboardLogout

Search

How to Get Involved

Take action with our streamlined project support process.

1

Select Loan

Choose a loan type to get started.

2

Quick Survey

Share your project goals and expected outcomes.

3

Eligibility Check

We assess impact and viability of your project.

4

Start Working on your Project

Receive funding and resources to execute your plan.



Loan Application

Loan Application
for Water Projects

Complete the fields below to apply for the loan

User ID *
test@gmail.com

Loan Category *
Water

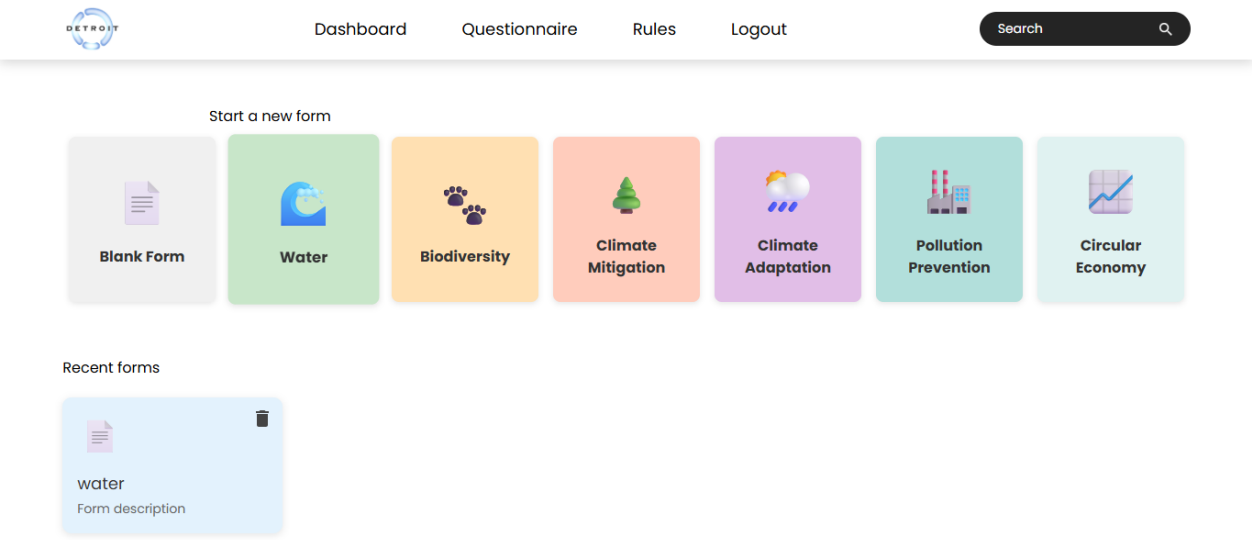
Amount *
₹ 15,00,00,000

Duration (Months) *
60
5 years

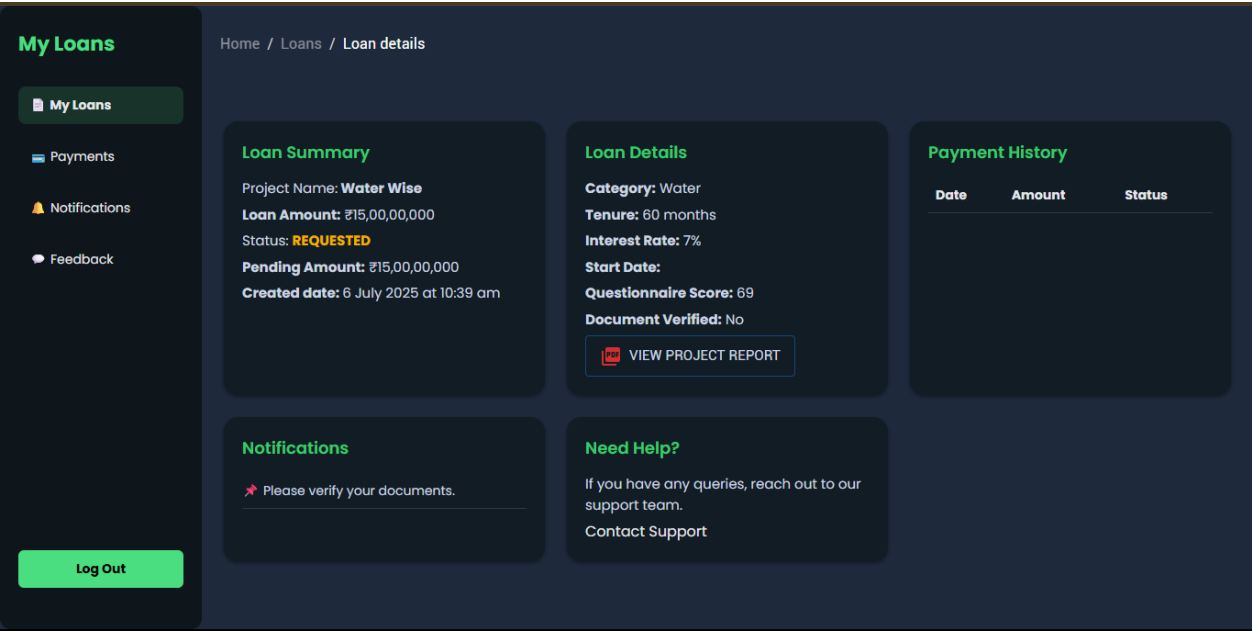
Project name *
Water Wise
Give a name to your project eg: My Water Project Loan

Project Report as PDF *
doc.pdf

Questionnaire Creating And Editing



Enterprise Dashboard



Feedback

My Loans

My Loans

Payments

Notifications

Feedback

Log Out

Submit Feedback

Please provide detailed information about your experience or suggestions.

Subject *

Brief description of your feedback

Details *

Provide detailed feedback...

• Minimum 10 characters

Attachments (Optional)

Click to attach files

By submitting, you agree we may use your input to improve our services. We will not share personal info.

* Required

Submit Feedback

localhost:5173/dashboard/feedback

Bank Dashboard with Loan Approving Process

Detroit SF

Dashboard

Loans

Payments

Feedback

Log Out

Loan Applications

Search by User ID, Name c

User ID	Name	Date	Status	Amount	Action
6	test 1	6 July 2025 at 10:41 am	REQUESTED	150000000	+
Eligibility: Not Eligible Score: 69% Documents: VIEW PROJECT REPORT Loan Amount 150000000 Duration Months 60 5 years Reject Approve					
5	Muhammed Aslam	21 June 2025 at 5:38 pm	DISBURSED	150000	-

Amount Dispersing After Approving the Loan

Detroit SF

Dashboard

Loans

Payments

Feedback

Log Out

Manage approved and disbursed loans

Q Search loans...

All Status

Refresh

Customer ↑↓	Loan Amount ↑↓	Status	Disbursed	Pending	Category
Muhammed Aslam testbrall@gmail.com ID: 1	\$150,000.00 7% APR + 5 months	DISBURSED	\$0.00 0.0%	\$150,000.00	Water loan gou

Disbursement Management

Disburse Amount

Enter amount

Maximum: \$150,000.00

Already Disbursed

\$0.00

Disbursement Progress

0.0% disbursed

Disburse

Remaining to Disburse

\$150,000.00

Loan Details

Loan Amount

\$150,000.00

Duration

5 months

Credit Score

100

Interest Rate

7% APR

Category

Water loan

Showing 1 loans

Rule Viewing

EU Taxonomy Sustainability Rules

Comprehensive database of environmental sustainability criteria

Q Search rules...

Circular Economy 21

Water 6

Climate Mitigation 101

Climate Adaptation 106

Pollution Prevention 6

Biodiversity 2

NACE	SECTOR	ACTIVITY #	ACTIVITY	CONTRIBUTION	DESCRIPTION
	Environmental protection		Conservation, including r		Initiation, development and
I55.10, I55.20, I55.30	Accommodation activiti		Hotels, holiday, camping		The provision of short-term

Showing 2 of 2 rules in Biodiversity

BIBLIOGRAPHY

10. BIBLIOGRAPHY

- <https://github.com>



- <https://chatgpt.com>



- <https://scholar.google.com>



- <https://stackoverflow.com>

